

РЕФЕРАТ

Отчет 62 с., 61 рис., 24 источн.

ИНТЕРНЕТ, РАЗРАБОТКА АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ, ВЕБ-ПРИЛОЖЕНИЕ, GOLANG, GRAPHQL, DOCKER, POSTGRESQL, GRPC, ЗДОРОВЬЕ, МЕДИЦИНА, УЧЕТ МЕДИЦИНСКИХ ЗАПИСЕЙ

Объект исследования – веб-приложение «MedApp».

Цель работы – разработка архитектуры приложения для мониторинга здоровья и ведения медицинских записей.

В ходе работы был проведен анализ предметной области, обзор веб-приложений с аналогичной тематикой, обоснование выбранной архитектуры.

С учетом постоянной потребности в медицинских услугах для ведения учета здоровья и записей к специалистам, в перспективе ожидается постоянный рост числа пользователей веб-приложения, с которым, благодаря архитектуре, справится разработанное приложения.

REPORT

Report 62 p., 61 figures, 24 sources.

INTERNET, APPLICATION ARCHITECTURE DEVELOPMENT, WEB APPLICATION, GOLANG, GRAPHQL, DOCKER, POSTGRESQL, GRPC, HEALTH, MEDICINE, MEDICAL RECORDS ACCOUNTING

The object of the study is the web application "MedApp".

The purpose of the work is to develop an application architecture for health monitoring and medical records management.

In the course of the work, an analysis of the subject area, an overview of web applications with similar topics, and a justification of the chosen architecture were carried out.

Taking into account the constant need for medical services for maintaining health records and records to specialists, in the future, a constant increase in the number of users of the web application is expected, which, thanks to the architecture, the developed application will cope with.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ.....	8
1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ	9
1.1 Анализ предметной области	9
1.2 Функциональные требования к системе.....	10
1.3 Нефункциональные требования к системе.....	11
1.4 Ограничения системы.....	11
2 ОБЗОР АРХИТЕКТУРНЫХ СТИЛЕЙ	13
2.1 Оценка архитектурных стилей	13
3 РАЗРАБОТКА АРХИТЕКТУРЫ.....	16
3.1 Основные компоненты системы.....	16
3.2 Взаимодействие между компонентами.....	16
3.3 Сценарии взаимодействия с компонентами системы	17
3.4 Модель базы данных.....	21
3.5 Физическая инфраструктура.....	23
4 ОЦЕНКА ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ.....	24
5 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ.....	29
5.1 Серверная часть.....	29
5.2 Клиентская часть.....	30
5.3 Технологии передачи данных.....	31
5.4 Базы данных.....	32
5.5 Разворачивание.....	33
6 РАЗРАБОТКА И РАЗВОРАЧИВАНИЕ	35
6.1 Разработка сервиса AuthService.....	35

6.2 Разработка сервиса HealthService	38
6.3 Разработка сервиса NoteService	40
6.4 Разработка сервиса CrewService	41
6.4 Разработка сервиса ApiGateway	42
6.5 Разработка клиента	45
6.6 Разворачивание.....	47
7 ТЕСТИРОВАНИЕ	52
ЗАКЛЮЧЕНИЕ	59
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	60

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяются следующие термины с соответствующими определениями:

API-шлюз	— компонент, выполняющий роль единой точки входа в микросервисную архитектуру, управляющий запросами, маршрутизацией и аутентификацией
Библиотека	— набор функций, классов или модулей, используемых разработчиками для решения типовых задач
Клиент	— программное обеспечение или устройство, запрашивающее ресурсы или услуги у сервера в сети
Микросервис	— независимый модуль системы, выполняющий определённую бизнес-логику
Монолит	— архитектура, где вся система построена как единое приложение с общей кодовой базой
Паттерн	— повторяемое решение типичной задачи проектирования в программировании
Репликация	— процесс копирования и синхронизации данных между несколькими серверами или базами данных для повышения доступности и отказоустойчивости
Фреймворк	— программная платформа, предоставляющая готовую структуру и инструменты для разработки приложений
Шардинг	— метод горизонтального разделения данных, при котором большие базы данных делятся на более мелкие части

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяются следующие сокращения и обозначения:

СУБД	– система управления базами данных
AMQP	– Advanced Message Queuing Protocol
API	– Application Programming Interface
REST	– Representational State Transfer
SOA	– Service-Oriented Architecture
SQL	– Structured Query Language
UML	– Unified Modeling Language
VPS	– Virtual Private Server

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка архитектуры и приложения для мониторинга здоровья и ведения медицинских записей, анализ предметной области и формирование ключевых требований к приложению. Проект предполагает разработку архитектуры и функциональности приложения с использованием языка программирования Golang [1], фреймворка Gin [2] и базы данных PostgreSQL [3]. Для тестирования будет использоваться ручное и модульное тестирование, а для контейнеризации и разворачивания Docker [4].

Для достижения данной цели были поставлены следующие задачи:

- 1) проанализировать предметную область разрабатываемого приложения,
- 2) выбрать инструменты для реализации серверной части,
- 3) разработать архитектуру и приложение с применением выбранных технологий,
- 4) провести тестирование приложения.

Для достижения поставленных задач используются методы анализа и сравнительного исследования. Результаты данной работы могут быть полезными для разработчиков приложений.

Информационной базой в данной работе являются знания, полученные в ходе выполнения практических работ в течение курса «Архитектура клиент-серверных приложений», а также интернет-ресурсы.

В разделе с описанием предметной области расположена основная информация о проведенном анализе предметной области, функциональные и нефункциональные требования к системе, а также ограничения системы.

В разделе обзора архитектурных стилей указаны сведения об оценке архитектурных стилей.

В разделе разработки архитектуры описаны основные компоненты системы, связи между ними, схемы и модели данных компонентов.

1 ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

Анализ предметной области проводился среди приложений на тематику «Мониторинга здоровья и ведения медицинских записей». Были выбраны интернет-ресурсы: ЕМИАС [5], SmartMed [6] и MedScannet [7], представленные на рисунках 1-3.

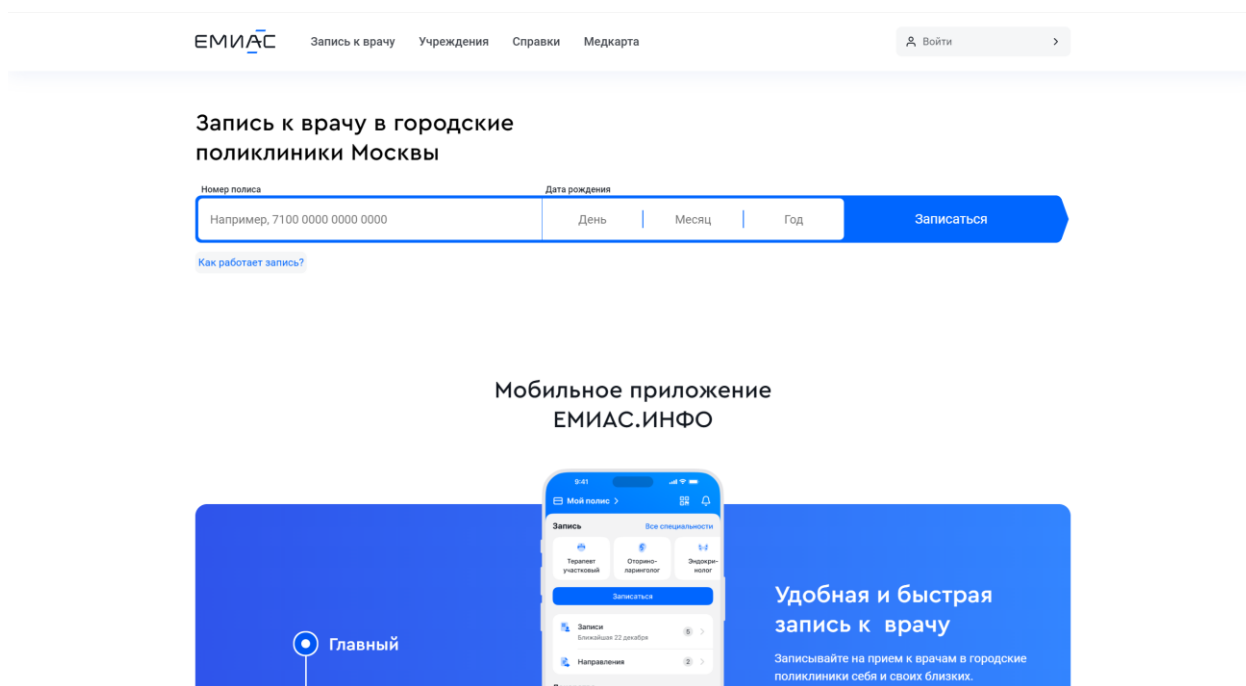


Рисунок 1 – Клиентская часть сайта ЕМИАС

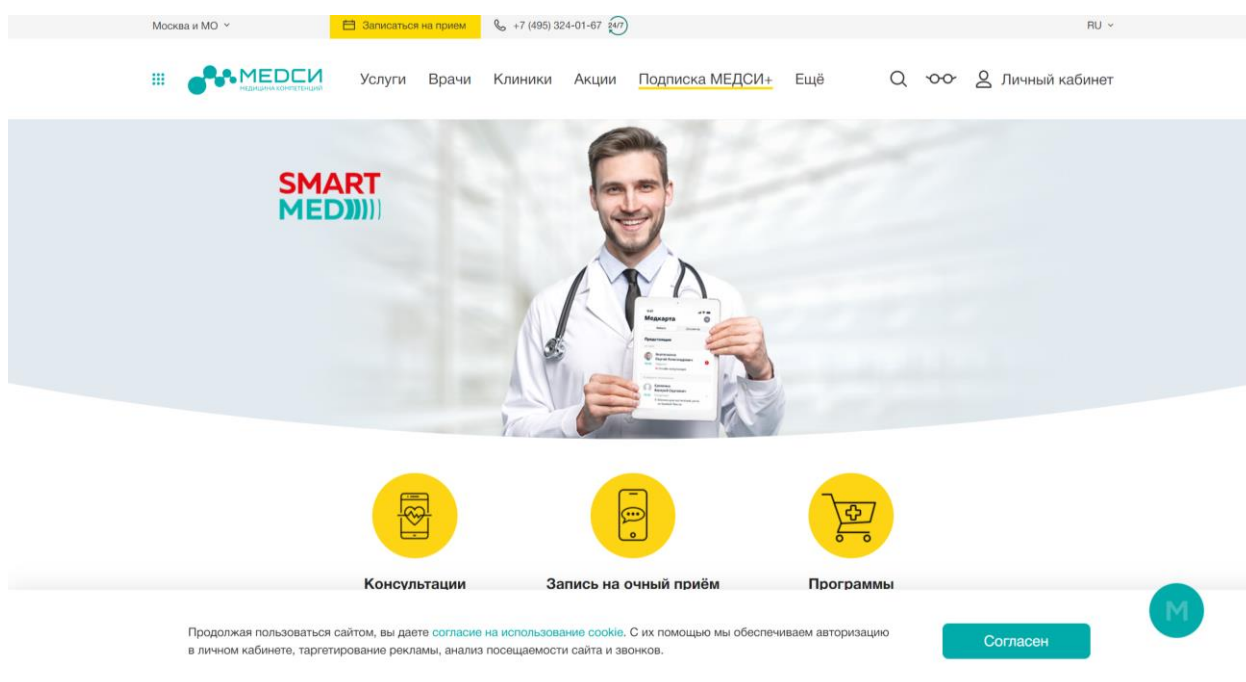


Рисунок 2 – Клиентская часть сайта SmartMed

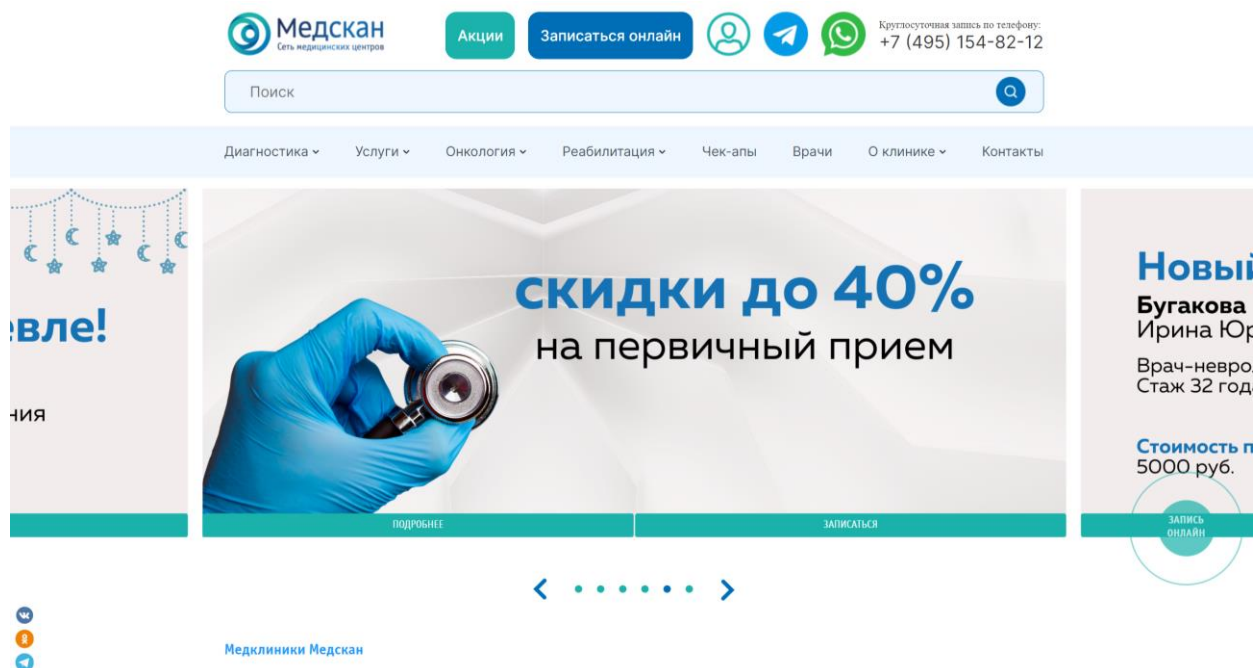


Рисунок 3 – Клиентская часть сайта Medscannet

Все перечисленные веб-приложение имеют регистрацию и вход в аккаунт, возможность ведения профиля пользователя, а также хранения записей к специалистам и доступа к личной статистике.

Исходя из анализа данных веб-приложений следует отметить, что разрабатываемый веб-ресурс данной тематики должен обладать следующими возможностями:

- регистрация нового аккаунта,
- вход в существующий аккаунт,
- выход из аккаунта,
- ведение профиля пользователя,
- возможность добавления записей к специалистам,
- просмотр и анализ статистики здоровья.

1.2 Функциональные требования к системе

Система мониторинга здоровья и ведения медицинских записей предоставляет пользователям возможность регистрации и управления своими медицинскими данными. Это включает создание профилей пациентов, где хранятся текущие записи к специалистам и личные медицинские данные. Пользователи смогут обновлять эти данные.

Система предусматривает доступ на основе ролей, разделяя функциональность между пациентами и врачами. На данный момент функционал для врачей отсутствует, но в дальнейшем планируется интегрировать его в работу приложения.

1.3 Нефункциональные требования к системе

Важнейшим аспектом является соответствие стандартам защиты персональных данных. Это подразумевает, что доступ к данным будет возможен только после аутентификации и авторизации пользователей.

Система должна обеспечивать высокую скорость работы, обрабатывая запросы с минимальными задержками и предоставляя доступ к медицинским записям в режиме реального времени. Это особенно важно при одновременном использовании системы несколькими пользователями.

Возможность масштабирования системы имеет решающее значение. Она должна легко адаптироваться к увеличению количества пользователей и объёма данных. Это позволит расширять возможности системы при добавлении новых функций, не влияя на её производительность.

Система должна быть устойчива к сбоям и продолжать функционировать даже при отказе отдельных компонентов. Для этого необходимо резервирование данных и распределённое управление процессами. Регулярное тестирование поможет выявлять проблемы на ранних этапах, что повысит надёжность системы и сократит время простоя.

1.4 Ограничения системы

Ограничения системы касаются поддерживаемых форматов и единиц измерения, интеграций, доступа по ролям.

Кроме того, система предусматривает ограничения доступа в зависимости от роли пользователя. Пациенты смогут редактировать только свои записи, а врачи – только записи своих пациентов.

С точки зрения технических ограничений, система будет функционировать в условиях ограниченных серверных ресурсов, таких как

оперативная память и процессорное время. Это потребует оптимизации алгоритмов и минимизации нагрузки для поддержания стабильности.

Пропускная способность сети также может ограничивать передачу данных в реальном времени, особенно если система используется на слабых или удаленных серверах.

2 ОБЗОР АРХИТЕКТУРНЫХ СТИЛЕЙ

2.1 Оценка архитектурных стилей

Существует множество архитектурных стилей, каждый из которых имеет свои особенности, преимущества и недостатки.

Одним из самых распространенных является монолитная архитектура [9]. Она представляет собой единую систему, в которой все компоненты и функции приложения объединены в одном проекте. Ключевыми преимуществами монолита являются простота в разработке и развертывании, так как все части приложения находятся в одном месте, минимальные затраты и сроки на разработку, а также легкость в тестировании, поскольку система тестируется как единое целое. Тем не менее, у монолитной архитектуры есть и недостатки, включая масштабирование. Для обработки увеличивающейся нагрузки необходимо масштабировать всю систему, а не отдельные компоненты. Это создает проблемы с обновлением, так как изменения в одном компоненте могут повлиять на всю систему, увеличивая риск ошибок и сбоев, а также ограничивает гибкость, так как различные технологии и языки программирования не могут быть использованы в рамках одного проекта.

Следующим архитектурным стилем является клиент-серверная архитектура [9]. В данном стиле приложение разделяется на две основные части: клиент, который запрашивает услуги, и сервер, который их предоставляет. Преимущества клиент-серверной архитектуры заключаются в четком разделении ролей между клиентом и сервером, что упрощает поддержку и развитие. Однако система имеет и свои недостатки, такие как зависимость от сети и количестве запросов на сервер только с клиента, что может негативно сказаться на производительности.

Сервисно-ориентированная архитектура (SOA) [10] представляет собой стиль, в котором приложение строится как набор взаимосвязанных сервисов, каждый из которых выполняет определенную функцию. SOA позволяет повторно использовать компоненты, что снижает затраты на разработку, и обеспечивает гибкость в изменении и масштабировании отдельных сервисов.

Однако управление взаимодействием между сервисами может быть сложным, и это требует значительных усилий для обеспечения совместимости всего приложения.

В последнее время все большую популярность приобретает микросервисная архитектура [11], которая является дальнейшим развитием SOA. Она предполагает, что приложение делится на множество небольших, самостоятельных сервисов, каждый из которых отвечает за свою сферу деятельности. Микросервисы обеспечивают высокую гибкость и возможность использования различных технологий и языков программирования для каждого сервиса. Также микросервисы упрощают развертывание и обновление, поскольку команды могут разворачивать и обновлять сервисы независимо друг от друга. Однако такое разнообразие сервисов может усложнить управление системой и вызвать трудности с тестированием и отладкой.

В контексте разработки приложения для мониторинга здоровья и ведения медицинских записей выбор микросервисной архитектуры обоснован несколькими ключевыми факторами. В сфере здравоохранения необходимо быстро адаптироваться к изменениям в бизнес-требованиях и объемах данных. Микросервисы позволяют независимо масштабировать различные части системы, такие как модули для обработки данных о пациентах или записи на прием к врачу, в зависимости от нагрузки.

Такая архитектура обеспечивает разделение ответственности, что позволяет различным командам сосредоточиться на разработке и поддержке отдельных сервисов. Это особенно полезно в медицинских приложениях, где различные аспекты требуют специализированного подхода.

Кроме того, микросервисная архитектура позволяет использовать разные технологии для разных сервисов, что дает возможность выбирать наиболее подходящие инструменты для каждой задачи. Это может улучшить производительность и поддержку. Еще одним важным преимуществом является устойчивость к сбоям, так как если один из микросервисов выходит

из строя, это не влияет на работоспособность всей системы, и важные бизнес-функции могут продолжать работать, обеспечивая пользователям доступ к необходимым услугам.

3 РАЗРАБОТКА АРХИТЕКТУРЫ

3.1 Основные компоненты системы

Основные компоненты системы включают в себя следующие элементы:

1) клиентское приложение, предоставляющее пользователям доступ к функциональности системы. Оно позволяет пользователям взаимодействовать с функционалом системы, например, регистрироваться, входить в личный кабинет, управлять медицинскими данными, записываться на прием к врачу;

2) API-шлюз выполняет роль единой точки входа для всех клиентских запросов к микросервисам системы. Он служит промежуточным слоем между клиентским приложением и внутренними сервисами, улучшая управление потоками данных;

3) сервис аутентификации, отвечающий за управление учетными записями пользователей, аутентификацию и авторизацию. Он обеспечивает безопасность данных и контроль доступа к системе;

4) сервис управления медицинскими данными, который обрабатывает все операции, связанные с медицинскими записями, такими как создание, обновление и хранение данных о пациентах;

5) сервис управления записями к врачам, отвечающий за организацию, планирование и обработку записей пользователей на прием к врачам, что способствует упрощению процесса обращения пациентов за медицинской помощью;

6) сервис управления персоналом, предназначенный для управления данными о медицинском персонале, осуществление добавления записи в базу данных к врачу;

7) базы данных, для хранения всей информации отдельно для каждого сервиса.

3.2 Взаимодействие между компонентами

Взаимодействие между компонентами системы осуществляется через технологии GraphQL [12], gRPC [13] и RabbitMQ [14], что позволяет обеспечить эффективный обмен данными. На рисунке 4, в виде UML [15]

диаграммы компонентов, отображено взаимодействие компонентов системы друг с другом.

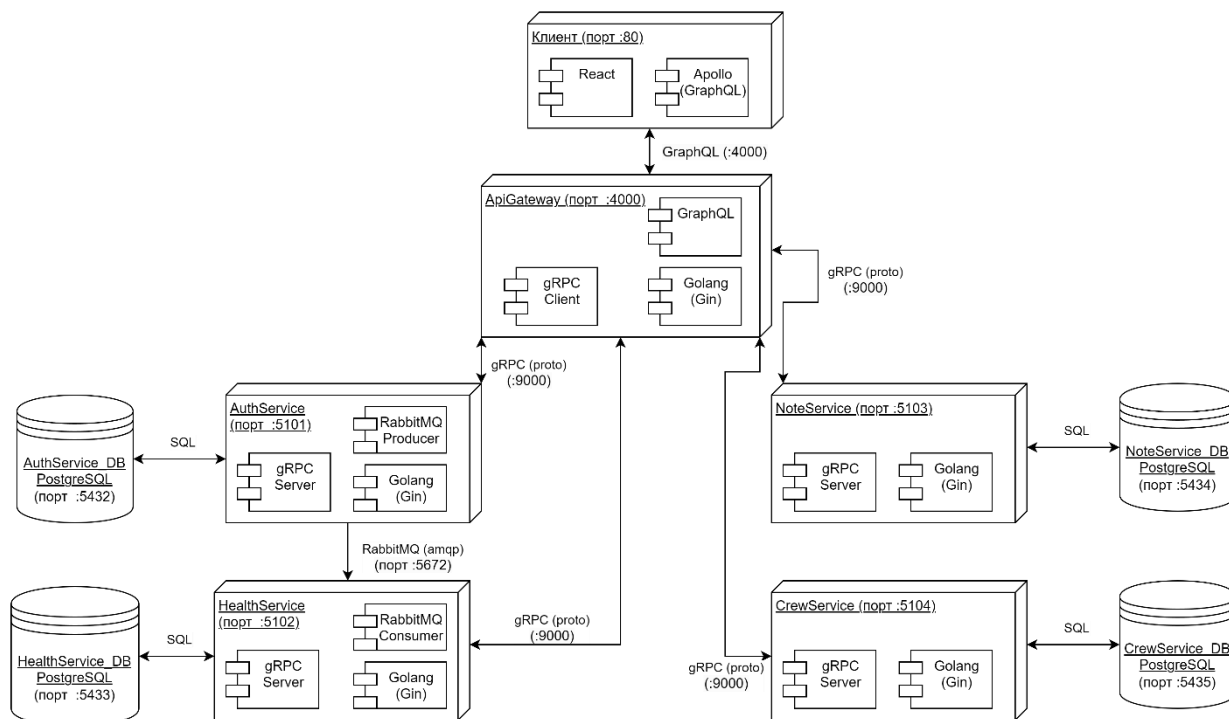


Рисунок 4 – Диаграмма компонентов системы

3.3 Сценарии взаимодействия с компонентами системы

Для пользования разрабатываемой системой, пользователь может различными способами взаимодействовать с компонентами системы. Различные сценарии поведения системы описаны с помощью диаграмм последовательности: вход в аккаунт (рисунок 5), регистрация аккаунта (рисунок 6), добавление медицинских данных в личный аккаунт (рисунок 7), запись к врачу (рисунок 8), получение всех своих записей (рисунок 9).

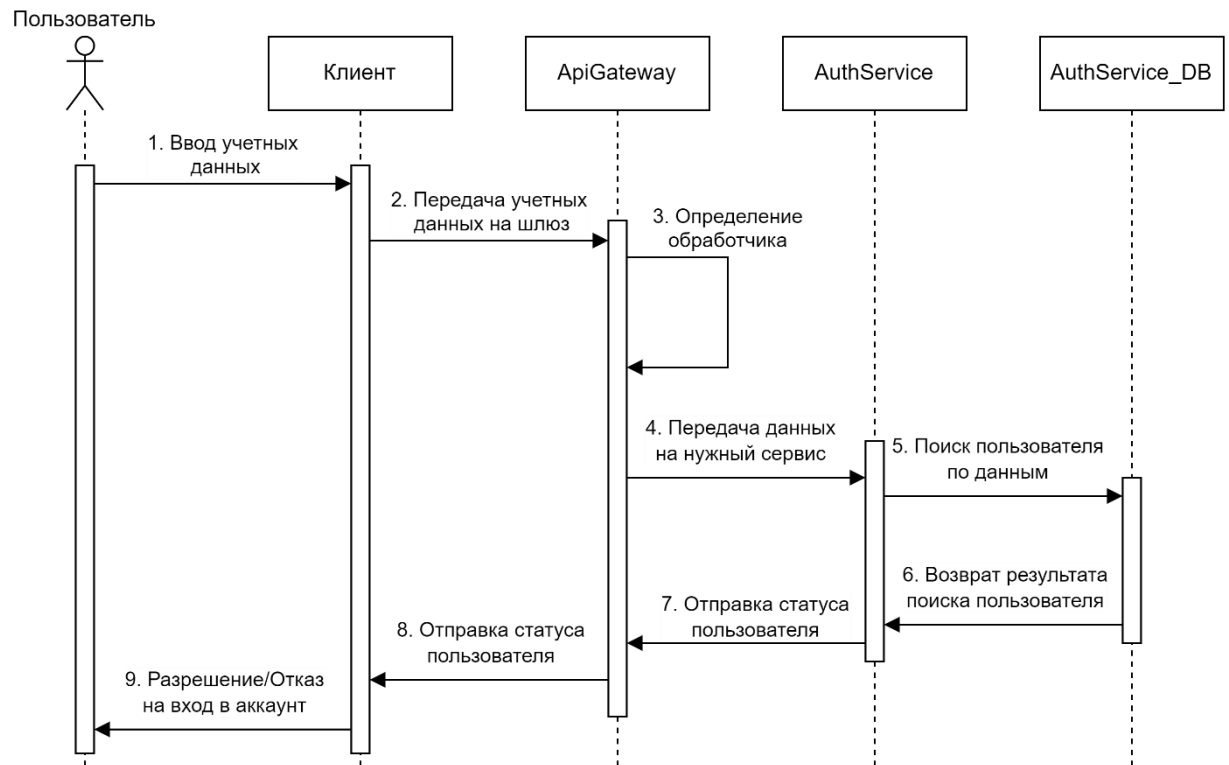


Рисунок 5 – Сценарий для входа в аккаунт

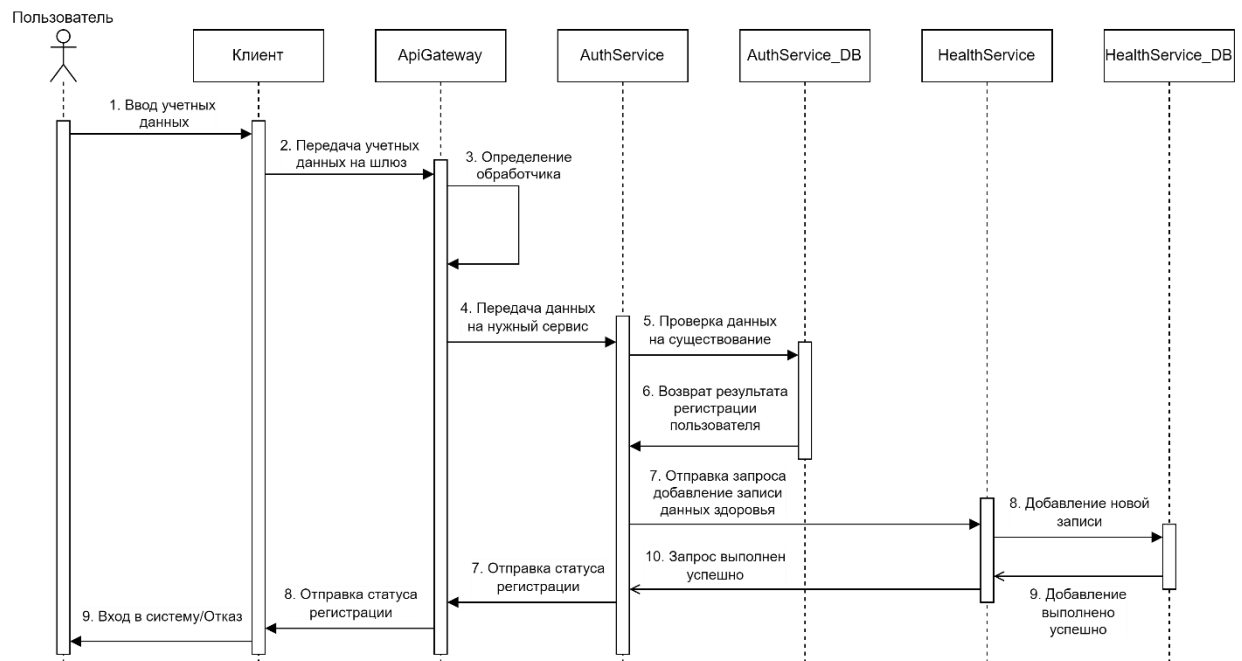


Рисунок 6 – Сценарий для регистрации аккаунта

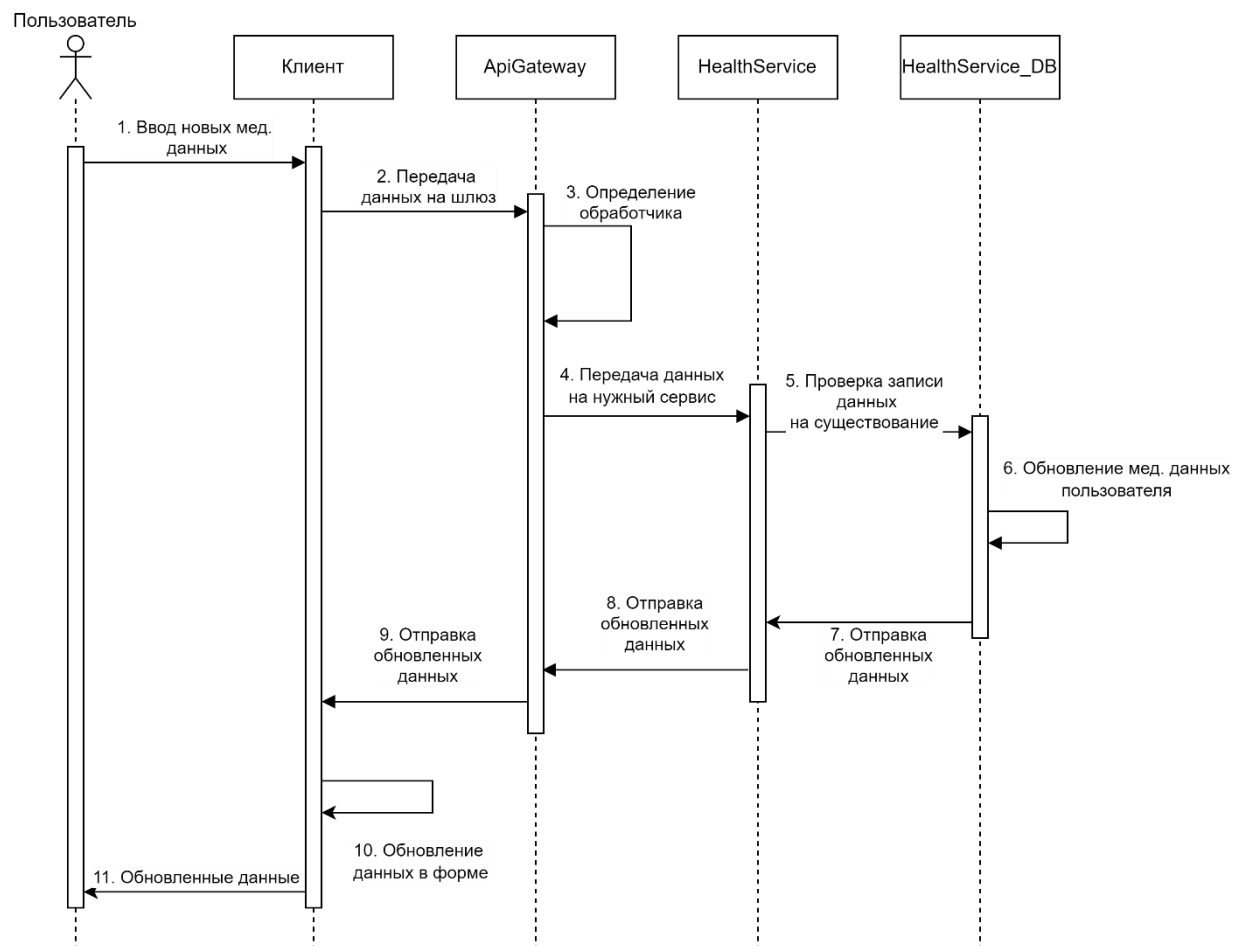


Рисунок 7 – Сценарий для добавления медицинских данных в личный аккаунт

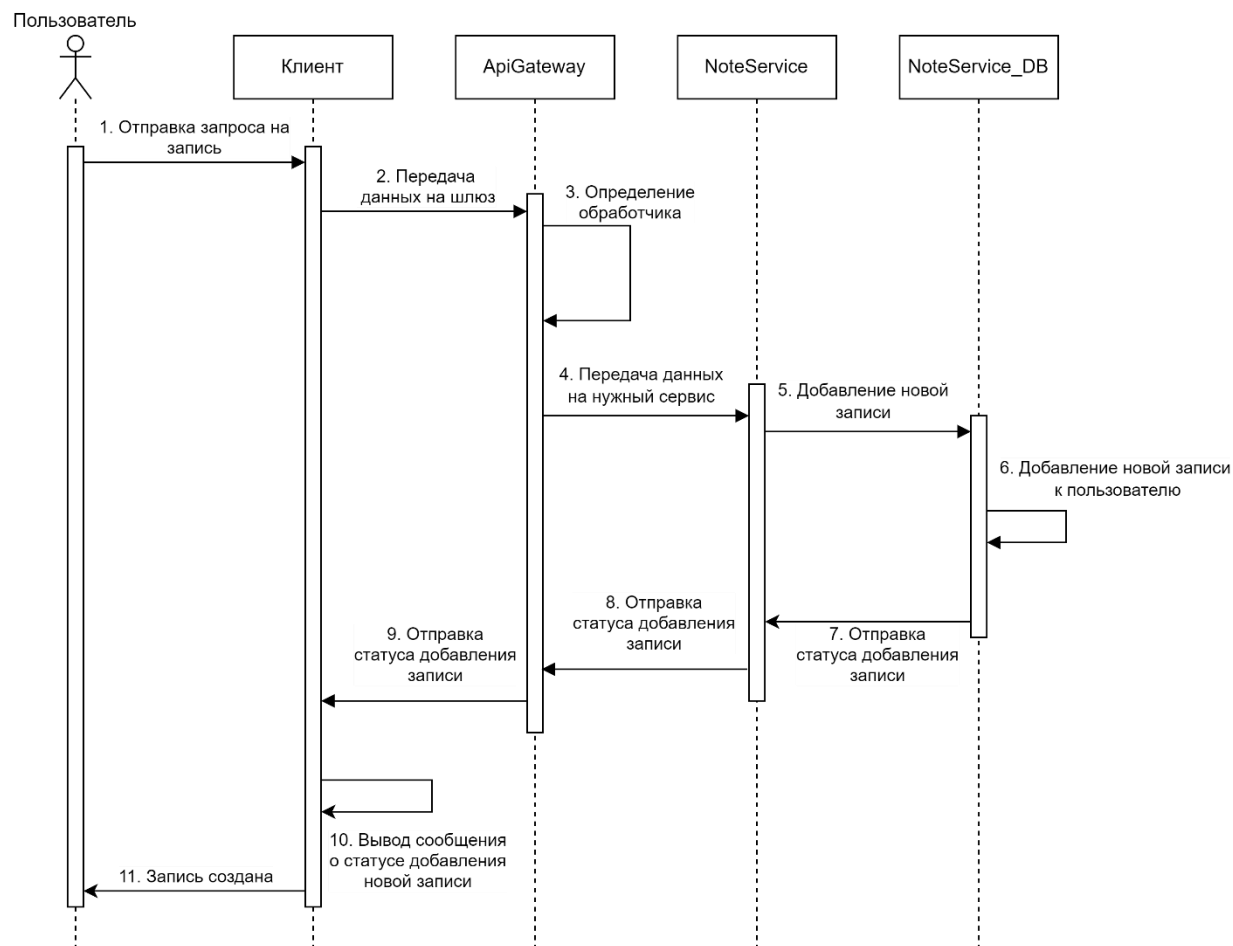


Рисунок 8 – Сценарий для записи к врачу

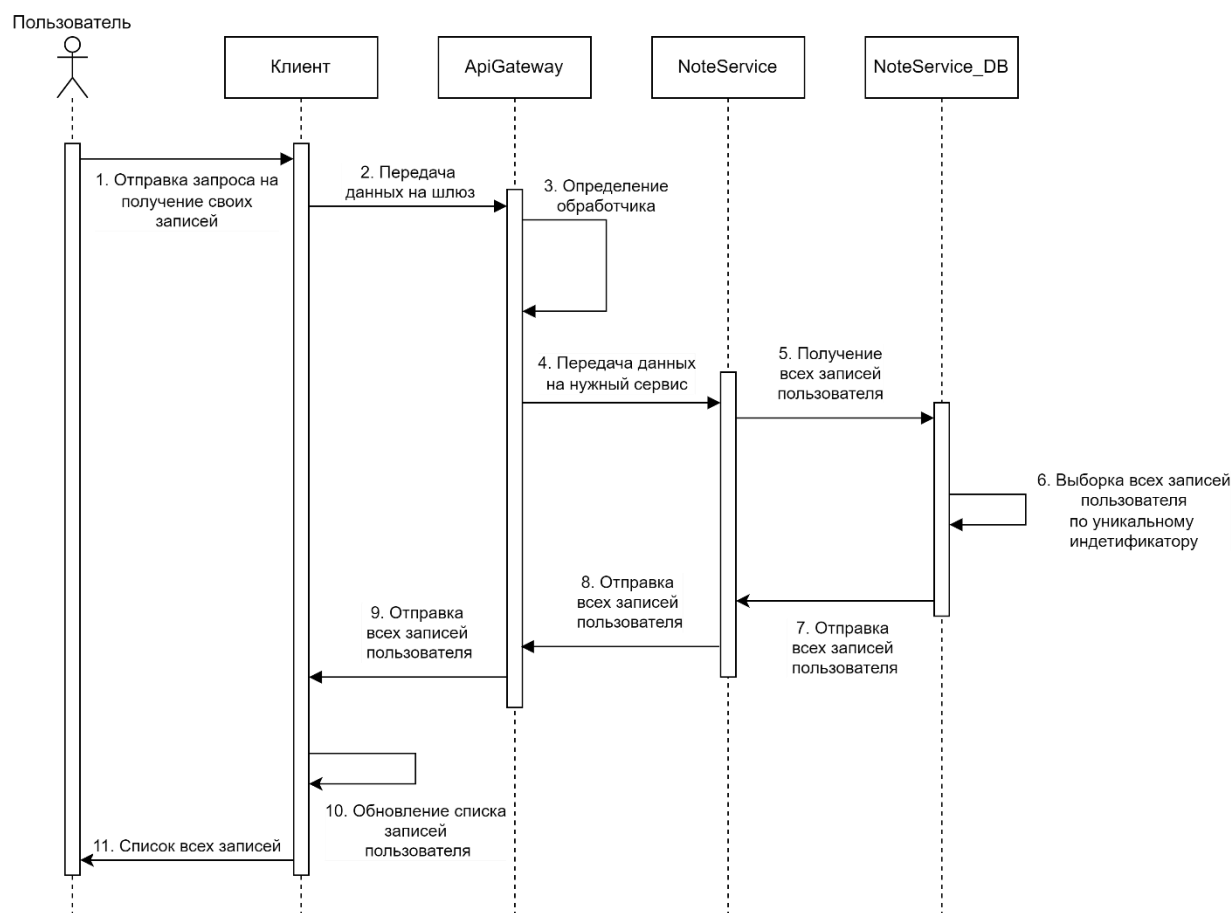


Рисунок 9 – Сценарий для получения всех своих записей

3.4 Модель базы данных

В системе реализована работа с базой данных с помощью СУБД PostgreSQL. Каждый сервис имеет свою собственную базу данных, что соответствует принципам микросервисной архитектуры. На рисунках 10–13 представлены модели баз данных для каждого сервиса, их структура, типы данных для каждого атрибута и связи между таблицами.

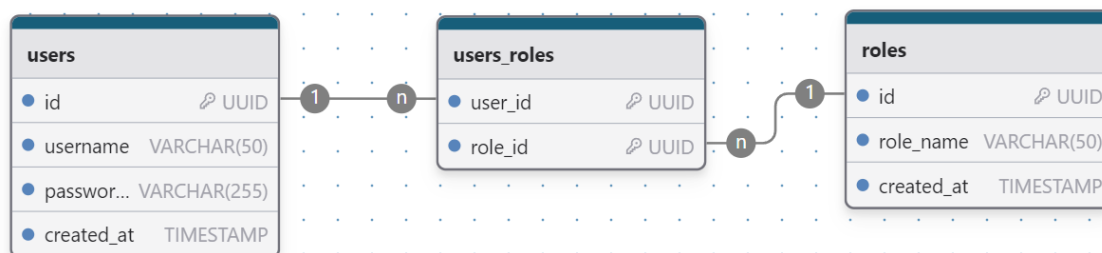


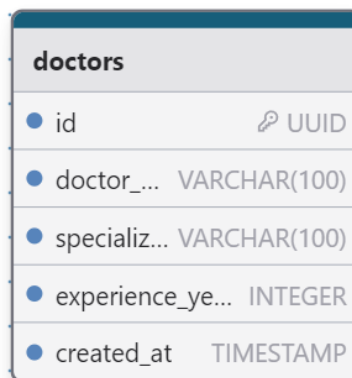
Рисунок 10 – Модель базы данных сервиса авторизации

users_health_data	
• userId	🔗 VARCHAR(100)
• age	INTEGER
• height	DECIMAL
• weight	DECIMAL
• pulse	INTEGER
• pressure	VARCHAR(30)
• created_at	TIMESTAMP

Рисунок 11 – Модель базы данных сервиса медицинских данных

appointments	
• id	🔗 UUID
• userId	VARCHAR(100)
• doctor_...	VARCHAR(100)
• specializ...	VARCHAR(100)
• visit_date	TIMESTAMP
• created_at	TIMESTAMP

Рисунок 12 – Модель базы данных сервиса записей к врачам



doctors	
• id	UUID
• doctor_...	VARCHAR(100)
• specializ...	VARCHAR(100)
• experience_ye...	INTEGER
• created_at	TIMESTAMP

Рисунок 13 – Модель базы данных сервиса управления персоналом

3.5 Физическая инфраструктура

Для разработки и развертывания приложения можно использовать различные типы физической инфраструктуры, такие как серверы под управлением операционных систем Linux или Windows, VPS или традиционные веб-хостинги.

Однако в рамках данного приложения планируется использовать метод контейнеризации с помощью Docker и Docker Compose [4], где для каждого сервиса будет сформирован отдельный образ самого сервиса и базы данных.

Контейнеризация предоставляет ряд преимуществ, в частности:

1) контейнеры можно легко перемещать между различными этапами разработки, при этом окружение и зависимости приложения остаются идентичными. Это снижает вероятность возникновения проблем, связанных с настройкой среды;

2) Docker и Docker Compose позволяют автоматизировать развертывание приложений с минимальными усилиями. Один файл конфигурации может описывать всю архитектуру приложения, включая необходимые контейнеры, их связи и параметры запуска, настройки сети.

4 ОЦЕНКА ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

Оценка паттернов проектирования в данном приложении выявляет ключевые преимущества и ограничения. В условиях использования микросервисной архитектуры предусмотрены определенные паттерны проектирования [16].

Паттерн Strangler, представленный на рисунке 14, предназначен для постепенного перехода к микросервисной архитектуре от монолитного приложения. Старая функциональность заменяется новыми сервисами, и последние используются после завершения работы, в то время как первые вытесняются.

Однако в разрабатываемой системе данный паттерн не имеет места быть, так как система изначально реализовывалась с упором на микросервисную архитектуру.



Рисунок 14 – Представление паттерна Strangler

Паттерн API-шлюз (ApiGateway), изображенный на рисунке 15, в данном случае играет важную роль в координации взаимодействий между клиентом и различными микросервисами, упрощая управление и обеспечивая единое точку доступа для запросов с клиента. ApiGateway также помогает с балансировкой нагрузки и, при необходимости, с авторизацией, повышая общую безопасность и управляемость. API-шлюз не предоставляет сервисы внешнему миру напрямую, чтобы обеспечить их безопасность. Все вопросы, связанные с безопасностью, решаются на шлюзе, и только аутентифицированные клиенты получают доступ.

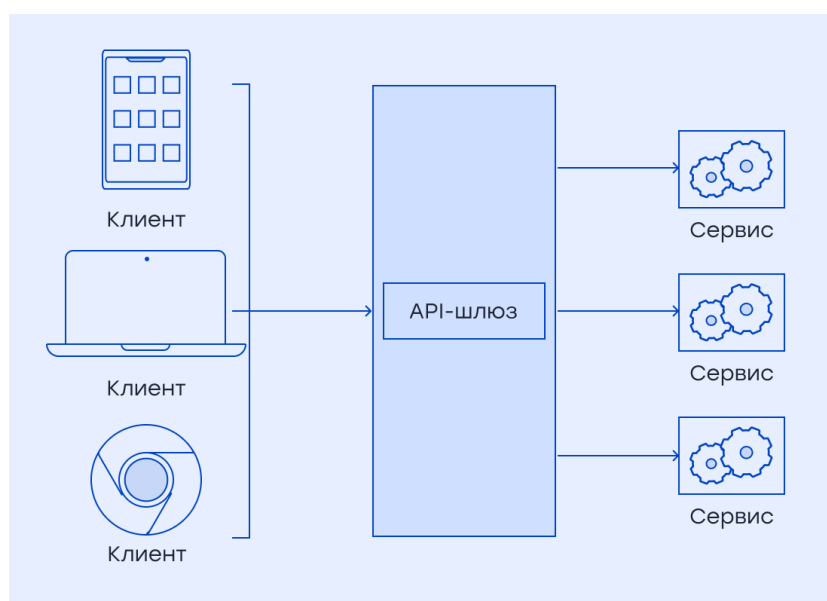


Рисунок 15 – Представление паттерна API-шлюз

Некоторые запросы требуют обращения к нескольким сервисам, что увеличивает время общения клиента с сервисами. Паттерн Агрегатор, представленный на рисунке 16, может общаться со всеми сервисами и возвращать ответ после агрегации, чтобы сократить обширное общение клиента с сервисами.

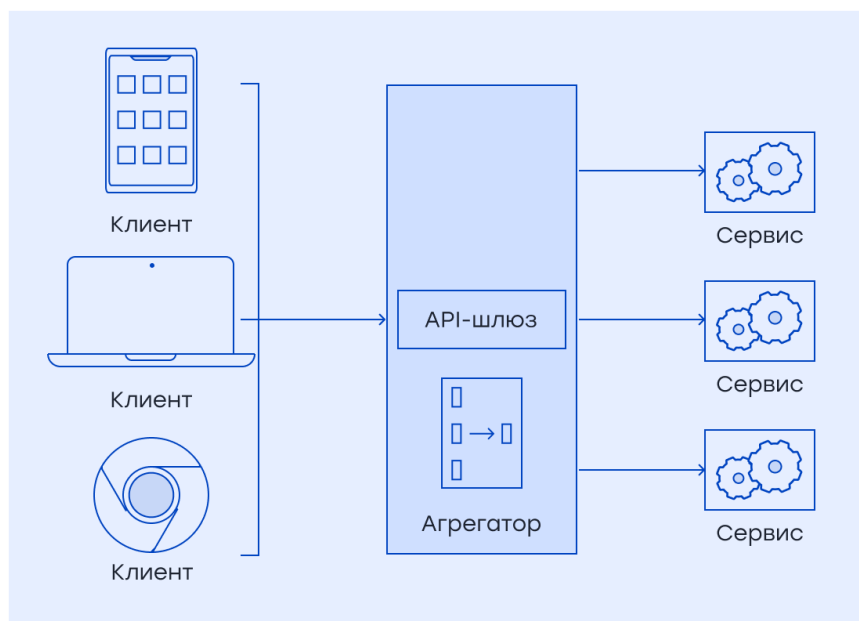


Рисунок 16 – Представление паттерна Агрегатор

Паттерн Saga, изображенный на рисунке 17, помогает в управлении транзакциями, где локальные транзакции в каждом сервисе (saga) выполняются, и выдают событие для следующего сервиса, чтобы начать транзакцию. Если какая-либо из транзакций терпит неудачу, серия транзакций, компенсирующих предыдущую транзакцию, будет выполнена saga, чтобы отменить все изменения, сделанные локальными транзакциями, предшествующими этой.

Существуют два способа реализации паттерна Saga: хореография и оркестрирование. В хореографии точка управления не централизована, что означает, что каждый сервис будет публиковать сообщение или событие для других сервисов, запуская локальную транзакцию.

Оркестрирование обеспечивает способ управления участниками saga, сообщая каждому сервису о локальной транзакции, которую ему необходимо выполнить. На событийной основе операции для saga и транзакций обрабатываются оркестратором saga. Состояние отдельной задачи управляется saga, и в случае сбоя он выполнит транзакцию, чтобы компенсировать предыдущие транзакции.

В разрабатываемой системе данный паттерн не используется, так как использование данного паттерна в текущей системе некорректно, избыточно с точки зрения самой архитектуры и расхода ресурсов системы.

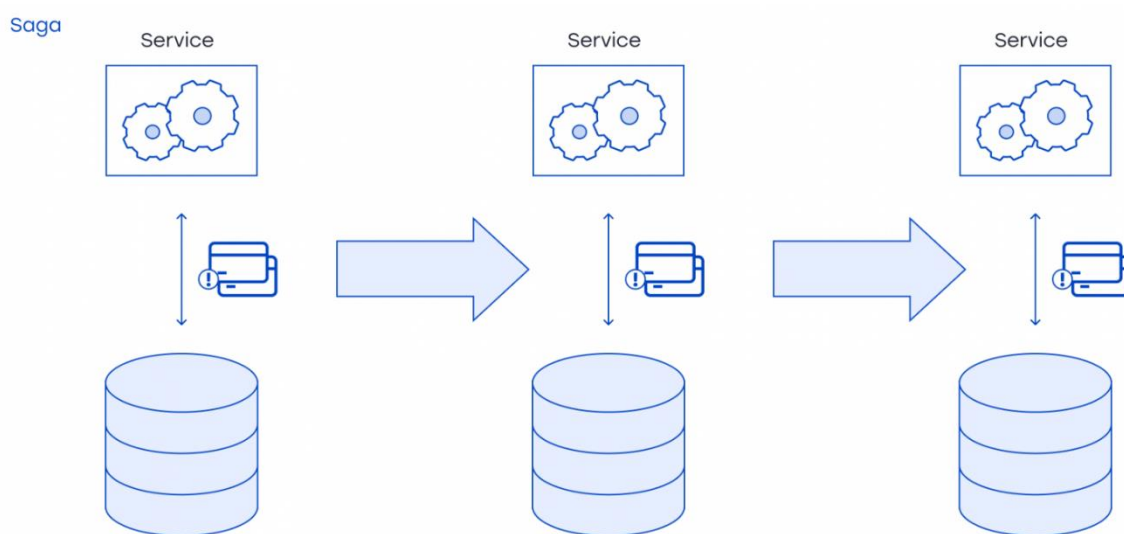


Рисунок 17 – Представление паттерна Saga

Паттерн «Асинхронный обмен сообщениями» весьма успешно используется в разрабатываемой системе, ведь некоторые запросы требуют взаимодействия между собой нескольких сервисов, и, чтобы минимизировать нагрузку и увеличить время отклика системы, используется данный паттерн. Для таких запросов должен быть предусмотрен асинхронный механизм с использованием служб брокера сообщений, например RabbitMQ. Пример применения данного паттерна представлен на рисунке 18.

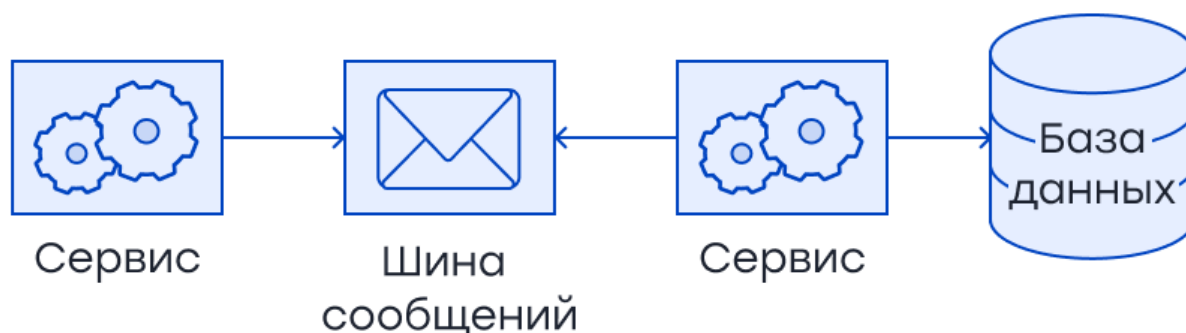


Рисунок 18 – Представление паттерна «Асинхронный обмен сообщениями»

Таким образом, проведя анализ известных паттернов, были выявлены несколько паттернов для разрабатываемой системы, чтобы улучшить

взаимодействие сервисов, быстроедействие, и дальнейшую масштабируемость, а именно паттерны: API-шлюз, Агрегатор и Асинхронный обмен сообщениями.

5 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

5.1 Серверная часть

При выборе технологий для реализации микросервисной архитектуры стоит рассмотреть несколько популярных языков программирования и их фреймворки, так как каждый язык обладает собственными преимуществами и недостатками для создания сложных распределенных систем.

Один из широко используемых языков для микросервисов – Java. Его ключевыми преимуществами являются зрелость экосистемы, обширная база библиотек и фреймворков, а также активное сообщество разработчиков. На Java доступен фреймворк Spring Boot [17], который предлагает инструменты для создания и управления микросервисами. Однако Java может быть избыточно ресурсоемкой, что приводит к высоким требованиям по памяти и производительности, особенно в условиях масштабирования.

Python также является популярным выбором для создания микросервисов. Его простота и высокая скорость разработки привлекают разработчиков. Для микросервисной архитектуры на Python часто используется фреймворк Django [18] или его более легковесная версия Flask, которая подходит для простых и небольших сервисов. Однако Python уступает в производительности и не предоставляет такой же высокой многозадачности, как другие языки, что ограничивает его возможности в сильно нагруженных системах.

Node.js [19] позволяет писать серверную логику на JavaScript и особенно хорошо подходит для высоконагруженных приложений с большим числом соединений. Такие фреймворки, как Express.js и Nest.js, предлагают хорошие возможности для создания микросервисов на основе событийной модели. Основным плюсом Node.js является его асинхронность и поддержка масштабирования, но данный набор технологий менее оптимален для процессорных задач и требует значительных усилий для обеспечения безопасности и устойчивости.

Golang (Go) [1], напротив, стал одним из предпочтительных языков для микросервисной архитектуры благодаря своей производительности, встроенной поддержке параллелизма и низким требованиям к ресурсам. Go был разработан с целью упрощения написания надежного, многопоточного кода, что делает его особенно подходящим для микросервисов, которые должны обрабатывать большое количество запросов в условиях минимальных задержек. Одним из популярных фреймворков для микросервисов на Go является Gin [2] – высокопроизводительный HTTP-фреймворк с минималистичным, но мощным набором функций. Gin обеспечивает быструю маршрутизацию, удобные инструменты для работы с запросами и поддерживает обработку ошибок, что делает его отличным выбором для построения API.

Сравнивая Golang с другими языками, можно выделить его простоту, высокую производительность и надежность, особенно в плане параллельной обработки запросов. Это особенно важно для микросервисов, которые должны быть независимыми, масштабируемыми и легко тестируемыми. Gin же обладает простым интерфейсом и позволяет сократить количество кода, необходимого для создания микросервисов. Так, комбинация Golang и Gin является оптимальным выбором для построения высокопроизводительных и масштабируемых микросервисов.

5.2 Клиентская часть

При выборе технологий для создания интерфейсов веб-приложений часто рассматриваются несколько популярных JavaScript-библиотек и фреймворков. Каждая из них имеет свои сильные и слабые стороны, влияющие на удобство разработки, производительность и возможности расширения.

Фреймворк Angular [20] предоставляет мощный набор инструментов для создания сложных, масштабируемых приложений с глубокой структурой. Angular предлагает полный набор функций, таких как маршрутизация, управление состоянием и инъекция зависимостей, что делает его идеальным для крупных проектов. Однако, из-за высокой сложности и наличия жестких

архитектурных требований, Angular требует более глубокого обучения, и его использование может быть избыточным для менее сложных приложений.

Vue.js [21] выделяется своей простотой и гибкостью. Этот фреймворк легко осваивается, имеет простой синтаксис и предоставляет достаточный набор инструментов для создания интерфейсов любой сложности. Vue также поддерживает двухстороннюю привязку данных и предоставляет возможности для расширения через плагины. Тем не менее, Vue менее распространен в крупных корпоративных приложениях, и у него нет большой экосистемы и сообщества, что иногда ограничивает доступные решения.

Библиотека React [22] стала одним из наиболее популярных выборов для создания интерфейсов, особенно в проектах с компонентной архитектурой. React предоставляет компонентный подход к построению интерфейсов, что упрощает управление состоянием и позволяет создавать модульные, переиспользуемые компоненты. Однако React представляет собой библиотеку, а не полноценный фреймворк, поэтому разработчики зачастую вынуждены дополнительно интегрировать инструменты для управления состоянием, маршрутизации и тестирования.

Сравнивая React с другими решениями, можно выделить его простоту, гибкость и широкую поддержку со стороны сообщества. Благодаря компонентной архитектуре и развитой экосистеме, React предоставляет разработчикам возможность быстро строить сложные пользовательские интерфейсы и поддерживать их в актуальном состоянии.

5.3 Технологии передачи данных

GraphQL [12] – это язык запросов, разработанный для более гибкого и эффективного взаимодействия между клиентом и сервером. В отличие от традиционного REST, где каждый запрос возвращает фиксированные данные, GraphQL позволяет клиенту запрашивать только те поля и данные, которые ему необходимы, что сокращает объем передаваемых данных и снижает нагрузку на сеть. Основными преимуществами GraphQL являются гибкость и

возможность создавать комплексные запросы, однако его настройка и обслуживание может быть сложнее относительно REST.

gRPC [13] – открытый фреймворк, ориентированный на высокопроизводительное взаимодействие между сервисами. gRPC использует Protocol Buffers для сериализации данных, что делает его легковесным и экономит пропускную способность. Одним из преимуществ gRPC является возможность работы в режиме реального времени, когда каждый сервис может отправлять и получать данные одновременно. Этот протокол особенно полезен для микросервисов, требующих высокой производительности и возможности многозадачности.

RabbitMQ [14] – брокер сообщений, который управляет асинхронной передачей данных между сервисами. В RabbitMQ сообщения помещаются в очереди, которые могут прослушивать другие сервисы, обрабатывая сообщения по мере их поступления. Основными преимуществами RabbitMQ являются его высокая стабильность, гибкость обмена сообщениями и возможность поддержки множества протоколов, например, AMQP.

5.4 Базы данных

При выборе базы данных для микросервисной архитектуры или любого масштабного приложения важно учитывать различные факторы: надежность, производительность, возможность масштабирования, поддержка транзакций.

PostgreSQL – это реляционная база данных с открытым исходным кодом, которая имеет отличную репутацию за свою надежность. PostgreSQL выделяется своей гибкостью, мощностью и широкой функциональностью, что делает её одним из лучших решений для современных приложений. Она поддерживает сложные SQL-запросы, транзакции и предоставляет обширный набор встроенных функций для работы с данными.

Сравнивая PostgreSQL с другими популярными базами данных, можно выделить её преимущества перед MySQL [23] и SQLite [24]. MySQL часто используется в веб-приложениях из-за её высокой скорости, но PostgreSQL обеспечивает более строгую поддержку стандартов SQL и предоставляет

дополнительные функции, такие как индексы на выражениях и возможности полнотекстового поиска, что делает её мощнее и гибче. В отличие от SQLite, которая является легковесной базой данных и подходит для небольших проектов или прототипов, PostgreSQL масштабируется и поддерживает работу с большими объемами данных, что делает её подходящей для крупных и высоконагруженных систем.

С точки зрения микросервисной архитектуры PostgreSQL также поддерживает шардинг и репликацию, что позволяет распределять нагрузку и обеспечивать высокую доступность. Это критично для приложений, которым требуется отказоустойчивость и возможность горизонтального масштабирования.

5.5 Разворачивание

Docker — платформа для контейнеризации, которая позволяет разработчикам упаковывать приложения и все их зависимости в изолированные контейнеры. Контейнеры представляют собой легковесные и независимые среды, содержащие код приложения, библиотеки, файлы конфигурации и другие зависимости, что делает их легкими для развертывания на различных системах. Контейнеры могут запускаться в изолированных пространствах на одном и том же сервере, благодаря чему достигается эффективное использование ресурсов.

Основное преимущество Docker заключается в том, что контейнеры делают приложения независимыми от конфигураций операционной системы и оборудования, что позволяет избежать распространенных проблем с совместимостью. Это значительно ускоряет процесс разработки, тестирования и развертывания приложений, особенно в условиях микросервисной архитектуры, где каждое приложение или его часть может быть развернуто в отдельном контейнере.

Кроме того, Docker позволяет эффективно масштабировать приложения. Поскольку контейнеры легковеснее виртуальных машин, они требуют меньше

ресурсов и быстрее запускаются, что делает Docker идеальным решением для микросервисной архитектуры.

6 РАЗРАБОТКА И РАЗВОРАЧИВАНИЕ

6.1 Разработка сервиса AuthService

После анализа предметной области, выбора подходящей архитектуры и ее составления, описания компонентов системы и сценариев взаимодействия с системой, выбора необходимых технологий, необходимо приступить к непосредственной реализации системы.

Начиная с серверной части приложения, реализуем основные сервисы нашей системы. Так как программный код сервисов весьма обширный, следовательно выделим основные части каждого сервиса.

На рисунках 19-23 представлена основная логика сервиса AuthService, отвечающего за авторизацию/аутентификацию пользователей, а также предоставление личной информации пользователя.

```
type IPasswordManager interface {
    CheckPasswordHash(password, hash string) error
}

You, 2 месяца назад | 1 author (You)
type passwordManager struct {
    minLen int
    log     logger.Logger
}

func NewPasswordManager(log logger.Logger) IPasswordManager {
    log.Info("PasswordManager created")
    return &passwordManager{minLen: 7, log: log}
}

func (p passwordManager) HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), 14)
    if err != nil {
        p.log.Errorf("Can't create hashed password with error: %v", err)
        return "", errors.New("managers-password: can't create hashed password")
    }

    p.log.Info("Password is hashed successfully")
    return string(bytes), nil
}

func (p passwordManager) ValidPassword(password string) error {
    if len(password) < p.minLen {
        p.log.Error("Password length less than minimum length")
        return errors.New("managers-password: user password is not valid: password length less than " + strconv.Itoa(p.minLen))
    }

    p.log.Info("Password is validated")
    return nil
}

func (p passwordManager) CheckPasswordHash(password, hash string) error {
    if err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password)); err != nil {
        p.log.Errorf("Password is not correct: %v", err)
        return errors.New("managers-password: password is not correct")
    }

    return nil
}
```

Рисунок 19 – Функционал парольного менеджера

```

type IRoleManager interface {
    // ...
}

var roles = []string{...}

You, 2 месяца назад | 1 author (You)
type roleManger struct {
    db *sql.DB
    log logger.ILogger
}

func NewRoleManager(db *sql.DB, log logger.ILogger) IRoleManager { ...
}

func (rm *roleManger) addRoleToUser(userId, role string) error { ...
}

func (rm *roleManger) AddRolesToUser(userId string, roles ...string) error {
    rm.log.Info("Role-Add: Transaction is beginning")
    tx, err := rm.db.Begin()
    if err != nil {
        rm.log.Errorf("Can't starts transaction: %v", err)
        return errors.New("managers-role: can't starts transaction for adding roles to user")
    }
    rm.log.Info("Role-Add: Transaction is begined successfully")
    defer tx.Rollback()

    for _, role := range roles {
        err := rm.addRoleToUser(userId, role)
        if err != nil {
            return err
        }
    }
    rm.log.Infof("All roles %v added to user %s successfully", roles, userId)

    if err = tx.Commit(); err != nil {
        rm.log.Errorf("Can't commits transaction: %v", err)
        return errors.New("managers-role: can't commits transaction for adding role to user")
    }

    rm.log.Info("Role-Add: Transaction is committed successfully")
    return nil
}

func (rm *roleManger) GetUserRole(userId string) (string, error) { ...
}

```

Рисунок 20 – Функционал менеджера ролей

```

func (rm *roleManger) deleteRoleFromUser(userId, role string) error {
    roleId, ok := rm.checkRoleExists(role)
    if !ok {
        return errors.New("managers-role: role " + role + " not exists in database")
    }

    query := `DELETE FROM users_roles WHERE user_id = $1 AND role_id = $2`
    _, err := rm.db.Exec(query, userId, roleId)
    if err != nil {
        rm.log.Errorf("Can't deletes role %s from user %s: %v", role, userId, err)
        return errors.New("managers-role: can't delete role " + role + " from the " + userId)
    }

    rm.log.Infof("Role %s deleted successfully from user %s", role, userId)
    return nil
}

You, 2 месяца назад • Change packages
func (rm *roleManger) DeleteRolesFromUser(userId string, roles ...string) error {
    rm.log.Info("Role-Delete: Transaction is beginning")
    tx, err := rm.db.Begin()
    if err != nil {
        rm.log.Errorf("Can't starts transaction: %v", err)
        return errors.New("managers-role: can't starts transaction for deleting roles from user")
    }
    rm.log.Info("Role-Delete: Transaction is begined successfully")
    defer tx.Rollback()

    for _, role := range roles {
        err := rm.deleteRoleFromUser(userId, role)
        if err != nil {
            rm.log.Errorf("Can't delete role %s from user %s", role, userId)
            return errors.New("managers-role: can't delete role " + role + " from user " + userId)
        }
    }
    rm.log.Infof("All roles %v deleted from user %s successfully", roles, userId)

    if err = tx.Commit(); err != nil {
        rm.log.Errorf("Can't commits transaction: %v", err)
        return errors.New("managers-role: can't commits transaction for deleting roles from user")
    }

    rm.log.Info("Role-Delete: Transaction is committed successfully")
    return nil
}

func (rm *roleManger) checkRoleExists(role string) (string, bool) { ...
}

```

Рисунок 21 – Функционал менеджера ролей пользователей

```

func (um *userManager) InsertUser(user UserRecord) (string, error) {
    var userId string
    query := `INSERT INTO users (username, password_hash) VALUES ($1, $2) RETURNING id`
    if err := um.db.QueryRow(query, user.UserName, user.Password).Scan(&userId); err != nil {
        um.log.Errorf("Can't adds new user (%s, %s) to the database: %v", user.UserName, user.Password, err)
        return "", errors.New("managers-user: can't creates user in the database")
    }

    um.log.Infof("User %s created successfully", userId)
    return userId, nil
}

func (um *userManager) GetUserByName(userName string) (UserRecord, error) {
    if userName == "" {
        um.log.Errorf("Can't gets an user (%s) from the database", userName)
        return UserRecord{}, errors.New("managers-user: can't get user from the database")
    }

    var user UserRecord
    query := `SELECT id, username, password_hash FROM users WHERE username = $1`
    if err := um.db.QueryRow(query, userName).Scan(&user.Id, &user.UserName, &user.Password); err != nil {
        um.log.Errorf("Can't gets an user (%s) from the database: %v", userName, err)
        return UserRecord{}, errors.New("managers-user: can't get user from the database")
    }

    um.log.Infof("User %s getted successfully", userName)
    return user, nil
}

func (um *userManager) GetUserById(userId string) (UserRecord, error) {
    if userId == "" {
        um.log.Errorf("Can't gets an user (%s) from the database", userId)
        return UserRecord{}, errors.New("managers-user: can't get user from the database")
    }

    var user UserRecord
    query := `SELECT id, username, password_hash FROM users WHERE id = $1`
    if err := um.db.QueryRow(query, userId).Scan(&user.Id, &user.UserName, &user.Password); err != nil {
        um.log.Errorf("Can't gets an user (%s) from the database: %v", userId, err)
        return UserRecord{}, errors.New("managers-user: can't get user from the database")
    }

    um.log.Infof("User %s getted successfully", userId)
    return user, nil
}

```

Рисунок 22 – Функционал менеджера пользователей

```

func (s *grpcServer) SignUp(ctx context.Context, in *pb.UserRequest) (*pb.AuthResponse, error) {
    if err := nil {
        return nil, err
    }

    userRole := "user"
    user := managers.UserRecord{UserName: in.Username, Password: hashedPassword}
    userId, err := s.userManager.InsertUser(user)
    if err != nil {
        return nil, err
    }

    err = s.roleManager.AddRolesToUser(userId, userRole)
    if err != nil {
        return nil, err
    }

    // publish message
    message := struct {
        UserId string `json:"userId"`
    }{UserId: userId}

    if err := s.rabbitMQServer.PublishMessage(message); err != nil {
        return nil, err
    }

    return &pb.AuthResponse{UserId: userId, Role: userRole}, nil
}

func (s *grpcServer) SignIn(ctx context.Context, in *pb.UserRequest) (*pb.AuthResponse, error) {
    user, err := s.userManager.GetUserByName(in.Username)
    if err != nil {
        return nil, err
    }

    if err := s.passwordManager.CheckPasswordHash(in.Password, user.Password); err != nil {
        return nil, err
    }

    userRole, err := s.roleManager.GetUserRole(user.Id)
    if err != nil {
        return nil, err
    }

    return &pb.AuthResponse{UserId: user.Id, Role: userRole}, nil
}

func (s *grpcServer) GetUserInfo(ctx context.Context, in *pb.UserInfoRequest) (*pb.UserInfoResponse, error) {

```

Рисунок 23 – Функционал сервера gRPC

6.2 Разработка сервиса HealthService

На рисунках 24-26 представлена основная логика сервиса HealthService, отвечающего за управление медицинскими записями пользователя.

```
func (hs *HealthService) InsertHealthData(userId string) error {
    if userId == "" {
        hs.log.Errorf("User id (%s) is empty", userId)
        return errors.New("health-services: user id is empty")
    }

    query := `INSERT INTO users_health_data (userId) VALUES ($1) ON CONFLICT (userId) DO NOTHING`
    if _, err := hs.db.Exec(query, userId); err != nil {
        hs.log.Errorf("Can't inserts new health data for user (%s) to database: %v", userId, err)
        return errors.New("health-services: can't inserts new health data to database")
    }

    hs.log.Info("New health data inserted successfully")
    return nil
}

func (hs *HealthService) GetHealthDataByUserId(userId string) (HealthData, error) {
    var healthData HealthData

    if userId == "" {
        hs.log.Errorf("User id (%s) is empty", userId)
        return healthData, errors.New("health-services: user id is empty")
    }

    query := `SELECT age, height, weight, pulse, pressure FROM users_health_data WHERE userId = $1`
    err := hs.db.QueryRow(query, userId).Scan(
        &healthData.Age,
        &healthData.Height,
        &healthData.Weight,
        &healthData.Pulse,
        &healthData.Pressure,
    )

    if err != nil {
        hs.log.Errorf("Can't gets a health data for user (%s) from database: %v", userId, err)
        return healthData, errors.New("health-services: can't gets a health data from database")
    }

    hs.log.Infof("User (%s) health data getted successfully", userId)
    return healthData, nil
}

func (hs *HealthService) UpdateHealthDataByUserId(userId string, healthData HealthData) (HealthData, error) {
    if userId == "" {
        hs.log.Errorf("User id (%s) is empty", userId)
        return healthData, errors.New("health-services: user id is empty")
    }
}
```

Рисунок 24 – Функционал компонента взаимодействия с базой данных

```

func NewConsumer(rmq rabbitmq.IRabbitMQServer, hs services.IHealthService, log logger.ILogger) IConsumer {
    log.Info("Consumer created")
    h := &consumer{
        rmq:      rmq,
        hs:        hs,
        log:       log,
        healthDataChan: make(chan json.RawMessage),
    }

    if err := h.rmq.ConsumeMessages(h.handleMessage); err != nil {
        log.Fatalf("Failed to start consuming messages: %v", err)
    }

    return h
}

func (h *consumer) handleMessage(message json.RawMessage) {
    h.healthDataChan <- message
}

func (h *consumer) ProcessMessages(ctx context.Context) error {
    h.wg.Add(1)
    defer h.wg.Done()

    for {
        select {
        case msg := <-h.healthDataChan:
            var messageData struct {
                UserId string `json:"userId"`
            }

            if err := json.Unmarshal(msg, &messageData); err != nil {
                h.log.Errorf("Failed to unmarshal message: %v", err)
                continue
            }

            if err := h.hs.InsertHealthData(messageData.UserId); err != nil {
                h.log.Errorf("Failed to insert health data: %v", err)
            }

        case <-ctx.Done():
            h.log.Info("Stopping message processing")
            return nil
        }
    }
}

```

Рисунок 25 – Функционал «консьюмера» RabbitMQ

```

func calcBodyMassIndex(weight float32, height float32) float32 {
    height /= 100
    return weight / (height * height)
}

func (s *grpcServer) GetHealthData(ctx context.Context, in *pb.GetHealthDataRequest) (*pb.HealthDataResponse, error) {
    healthData, err := s.healthService.GetHealthDataByUserId(in.UserId)
    if err != nil {
        return nil, err
    }

    return &pb.HealthDataResponse{
        Age:      healthData.Age,
        Height:   healthData.Height,
        Weight:   healthData.Weight,
        Pulse:    healthData.Pulse,
        Pressure: healthData.Pressure,
        DailyWater: calcDailyWater(healthData.Weight),
        BodyMassIndex: calcBodyMassIndex(healthData.Weight, healthData.Height),
    }, nil
}

func (s *grpcServer) UpdateHealthData(ctx context.Context, in *pb.UpdateHealthDataRequest) (*pb.HealthDataResponse, error) {
    oldHealthData := services.HealthData{
        Age:      in.Age,
        Height:   in.Height,
        Weight:   in.Weight,
        Pulse:    in.Pulse,
        Pressure: in.Pressure,
    }

    healthData, err := s.healthService.UpdateHealthDataByUserId(in.UserId, oldHealthData)
    if err != nil {
        return nil, err
    }

    return &pb.HealthDataResponse{
        Age:      healthData.Age,
        Height:   healthData.Height,
        Weight:   healthData.Weight,
        Pulse:    healthData.Pulse,
        Pressure: healthData.Pressure,
        DailyWater: calcDailyWater(healthData.Weight),
        BodyMassIndex: calcBodyMassIndex(healthData.Weight, healthData.Height),
    }, nil
}

```

Рисунок 26 – Функционал сервера gRPC

6.3 Разработка сервиса NoteService

На рисунках 27-29 представлена основная логика сервиса NoteService, отвечающего за управление записями пользователя к врачам.

```
You, 2 месяца назад | 1 author (You)
type INoteService interface {
    InsertUserNote(userId, doctorName, specialization, visitDate string) error
    GetUserNotes(userId string) ([]NoteData, error)
}

You, 2 месяца назад | 1 author (You)
type noteService struct {
    db *sql.DB
    log logger.ILogger
}

You, 2 месяца назад | 1 author (You)
type NoteData struct {
    Id string
    Doctor_name string
    Specialization string
    Visit_date string
}

You, 2 месяца назад * Update note and crew services
func NewNoteService(db *sql.DB, log logger.ILogger) INoteService {
    log.Info("NoteService created")
    return &noteService{
        db: db,
        log: log,
    }
}

func (ns *noteService) InsertUserNote(userId, doctorName, specialization, visitDate string) error {
    if userId == "" {
        ns.log.Errorf("User id (%s) is empty", userId)
        return errors.New("note-services: user id is empty")
    }

    query := `INSERT INTO appointments (userId, doctor_name, specialization, visit_date) VALUES ($1, $2, $3, $4)`
    if _, err := ns.db.Exec(query, userId, doctorName, specialization, visitDate); err != nil {
        ns.log.Errorf("Can't inserts new note data for user (%s) to database: %v", userId, err)
        return errors.New("note-services: can't inserts new note data to database")
    }

    ns.log.Info("New health data inserted successfully")
    return nil
}
```

Рисунок 27 – Функционал управления записями

```
func (ns *noteService) InsertUserNote(userId, doctorName, specialization, visitDate string) error {
    ns.log.Errorf("Can't inserts new note data for user (%s) to database: %v", userId, err)
    return errors.New("note-services: can't inserts new note data to database")
}

ns.log.Info("New health data inserted successfully")
return nil
}

func (ns *noteService) GetUserNotes(userId string) ([]NoteData, error) {
    var notes []NoteData

    if userId == "" {
        ns.log.Errorf("User id (%s) is empty", userId)
        return notes, errors.New("note-services: user id is empty")
    }

    query := `SELECT id, doctor_name, specialization, visit_date FROM appointments WHERE userId = $1`
    rows, err := ns.db.Query(query, userId)
    if err != nil {
        ns.log.Errorf("Can't get notes data for user (%s) from database: %v", userId, err)
        return notes, errors.New("note-services: can't get notes data from database")
    }
    defer rows.Close()

    for rows.Next() {
        var noteData NoteData
        err := rows.Scan(
            &noteData.Id,
            &noteData.Doctor_name,
            &noteData.Specialization,
            &noteData.Visit_date,
        )
        if err != nil {
            ns.log.Errorf("Error scanning note data for user (%s): %v", userId, err)
            return notes, errors.New("note-services: error scanning note data")
        }
        notes = append(notes, noteData)
    }

    if err = rows.Err(); err != nil {
        ns.log.Errorf("Error occurred during rows iteration for user (%s): %v", userId, err)
        return notes, errors.New("note-services: error during rows iteration")
    }

    ns.log.Infof("User (%s) notes data retrieved successfully", userId)
    return notes, nil
}
```

Рисунок 28 – Функционал управления записями

```

func InitNoteGrpcServer(
    pb.RegisterVisitServiceServer(server.Engine, grpcServer)

    logger.Info("Initialize of grpc server successfully")
}

You, 2 месяца назад | 1 author (You)
type grpcServer struct {
    pb.UnimplementedVisitServiceServer

    noteService services.INoteService
    log          logger.ILogger
}

func (s *grpcServer) AddVisitRecord(ctx context.Context, in *pb.AddVisitRecordRequest) (*pb.AddVisitRecordResponse, error) {
    err := s.noteService.InsertUserNote(in.UserId, in.DoctorName, in.Specialization, in.VisitDate)
    if err != nil {
        return nil, err
    }

    return &pb.AddVisitRecordResponse{
        Success: true,
    }, nil
}

func (s *grpcServer) GetUserVisitRecords(ctx context.Context, in *pb.GetUserVisitRecordsRequest) (*pb.GetUserVisitRecordsResponse, error) {
    noteData, err := s.noteService.GetUserNotes(in.UserId)
    if err != nil {
        return nil, err
    }

    var visitRecords []*pb.VisitRecord
    for _, note := range noteData {
        visitRecord := &pb.VisitRecord{
            RecordNumber: note.Id,
            DoctorName:   note.Doctor_name,
            Specialization: note.Specialization,
            VisitDate:   note.Visit_date,
        }
        visitRecords = append(visitRecords, visitRecord)
    }

    return &pb.GetUserVisitRecordsResponse{
        VisitRecords: visitRecords,
    }, nil
}

```

Рисунок 29 – Функционал сервера gRPC

6.4 Разработка сервиса CrewService

На рисунках 30-31 представлена основная логика сервиса CrewService, отвечающего за управление данными персонала.


```

type crewService struct {
    db *sql.DB
    log logger.ILogger
}

// You, 2 месяца назад • Update note and crew services
func NewCrewService(db *sql.DB, log logger.ILogger) ICrewService {
    log.Info("CrewService created")
    return &crewService{
        db: db,
        log: log,
    }
}

func (cs *crewService) GetAllCrew() ([]CrewData, error) {
    var crews []CrewData

    query := `SELECT id, doctor_name, specialization, experience_years FROM doctors`
    rows, err := cs.db.Query(query)
    if err != nil {
        cs.log.Errorf("Can't get crews data from database: %v", err)
        return crews, errors.New("crew-services: can't get crews data from database")
    }
    defer rows.Close()

    for rows.Next() {
        var crewData CrewData
        err := rows.Scan(
            &crewData.DoctorId,
            &crewData.DoctorName,
            &crewData.Specialization,
            &crewData.ExperienceYears,
        )
        if err != nil {
            cs.log.Errorf("Error scanning crew data: %v", err)
            return crews, errors.New("crew-services: error scanning crews data")
        }
        crews = append(crews, crewData)
    }

    if err = rows.Err(); err != nil {
        cs.log.Errorf("Error occurred during rows iteration: %v", err)
        return crews, errors.New("crew-services: error during rows iteration")
    }

    return crews, nil
}

```

Рисунок 30 – Функционал парольного менеджера

```

func InitCrewGrpcServer(
    server *grpc.GrpcServer,
    logger logger.ILogger,
    crewService services.ICrewService) {
    grpcServer := &grpcServer{
        crewService: crewService,
        log:         logger,
    }
    pb.RegisterCrewServiceServer(server.Engine, grpcServer)
    logger.Info("Initialize of grpc server successfully")
}

// You, 2 месяца назад | 1 author (You)
type grpcServer struct {
    pb.UnimplementedCrewServiceServer

    crewService services.ICrewService
    log         logger.ILogger
}

func (s *grpcServer) GetDoctors(ctx context.Context, in *pb.GetDoctorsRequest) (*pb.GetDoctorsResponse, error) {
    crewData, err := s.crewService.GetAllCrew()
    if err != nil {
        return nil, err
    }

    var doctors []*pb.Doctor
    for _, crew := range crewData {
        doctor := &pb.Doctor{
            DoctorId:      crew.DoctorId,
            DoctorName:    crew.DoctorName,
            Specialization: crew.Specialization,
            ExperienceYears: int32(crew.ExperienceYears),
        }
        doctors = append(doctors, doctor)
    }

    return &pb.GetDoctorsResponse{
        Doctors: doctors,
    }, nil
}

```

Рисунок 31 – Функционал сервера gRPC

6.4 Разработка сервиса ApiGateway

На рисунках 32-35 представлена основная логика, а именно обработчики GraphQL, которые дальше взаимодействуют с сервисами с помощью gRPC, сервиса ApiGateway, отвечающего за передачу запроса с клиента на определенный сервис системы.

```
// SignUp is the resolver for the signUp field.
func (r *MutationResolver) SignUp(ctx context.Context, input model.AuthData) (*model.AuthPayload, error) {
    authConnIdx := findServiceIndex(r.GrpcServices, "auth_service")
    client := pbAuth.NewAuthServiceClient(r.GrpcServices[authConnIdx].Conn)

    response, err := client.SignUp(ctx, &pbAuth.UserRequest{
        Username: input.Username,
        Password: input.Password,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not sign up new user: %v", err)
    }

    return &model.AuthPayload{
        UserID: response.GetUserId(),
        Role:   response.GetRole(),
    }, nil
}

// SignIn is the resolver for the signIn field.
func (r *MutationResolver) SignIn(ctx context.Context, input model.AuthData) (*model.AuthPayload, error) {
    authConnIdx := findServiceIndex(r.GrpcServices, "auth_service")
    client := pbAuth.NewAuthServiceClient(r.GrpcServices[authConnIdx].Conn)

    response, err := client.SignIn(ctx, &pbAuth.UserRequest{
        Username: input.Username,
        Password: input.Password,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not sign in exists user: %v", err)
    }

    return &model.AuthPayload{
        UserID: response.GetUserId(),
        Role:   response.GetRole(),
    }, nil
}
```

Рисунок 32 – Обработчики запросов с клиента

```
// UpdateUserHealthData is the resolver for the updateUserHealthData field.
func (r *MutationResolver) UpdateUserHealthData(ctx context.Context, input model.UpdateUserHealthDataInput) (*model.UserAccountPayload, error) {
    authConnIdx := findServiceIndex(r.GrpcServices, "auth_service")
    authClient := pbAuth.NewAuthServiceClient(r.GrpcServices[authConnIdx].Conn)
    healthConnIdx := findServiceIndex(r.GrpcServices, "health_service")
    healthClient := pbHealth.NewHealthServiceClient(r.GrpcServices[healthConnIdx].Conn)

    responseAuth, err := authClient.GetUserInfo(ctx, &pbAuth.UserInfoRequest{
        UserID: input.UserID,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get user name: %v", err)
    }

    responseHealth, err := healthClient.UpdateHealthData(ctx, &pbHealth.UpdateHealthDataRequest{
        UserID: input.UserID,
        Age:     int32(*input.Age),
        Height:  float32(*input.Height),
        Weight:  float32(*input.Weight),
        Pulse:   int32(*input.Pulse),
        Pressure: *input.Pressure,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get user health data: %v", err)
    }

    return &model.UserAccountPayload{
        Username: responseAuth.GetUserName(),
        Age:      int(responseHealth.GetAge()),
        Height:   float64(responseHealth.GetHeight()),
        Weight:   float64(responseHealth.GetWeight()),
        Pulse:    int(responseHealth.GetPulse()),
        Pressure: responseHealth.GetPressure(),
        DailyWater: toFloat64Pointer(responseHealth.GetDailyWater()),
        BodyMassIndex: toFloat64Pointer(responseHealth.GetBodyMassIndex()),
    }, nil
}
```

Рисунок 33 – Обработчики запросов с клиента

```
// AddVisitRecord is the resolver for the addVisitRecord field.
func (r *mutationResolver) AddVisitRecord(ctx context.Context, input model.AddVisitRecordInput) (bool, error) {
    noteConnIdx := findServiceIndex(r.GrpcServices, "note_service")
    noteClient := pbNote.NewVisitServiceClient(r.GrpcServices[noteConnIdx].Conn)

    response, err := noteClient.AddVisitRecord(ctx, &pbNote.AddVisitRecordRequest{
        UserId:      input.UserID,
        DoctorName:   input.DoctorName,
        Specialization: input.Specialization,
        VisitDate:    input.VisitDate,
    })
    if err != nil {
        fmt.Println(err)
        return false, fmt.Errorf("graphql: could not add visit for name: %v", err)
    }
    // You, 2 месяца назад • Update crew and note services

    return response.GetSuccess(), nil
}

// GetUserInfo is the resolver for the getUserInfo field.
func (r *queryResolver) GetUserInfo(ctx context.Context, input model.UserAccountData) (*model.UserAccountPayload, error) {
    authConnIdx := findServiceIndex(r.GrpcServices, "auth_service")
    authClient := pbAuth.NewAuthServiceClient(r.GrpcServices[authConnIdx].Conn)
    healthConnIdx := findServiceIndex(r.GrpcServices, "health_service")
    healthClient := pbHealth.NewHealthServiceClient(r.GrpcServices[healthConnIdx].Conn)

    responseAuth, err := authClient.GetUserInfo(ctx, &pbAuth.UserInfoRequest{
        UserId: input.UserID,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get user name: %v", err)
    }

    responseHealth, err := healthClient.GetHealthData(ctx, &pbHealth.GetHealthDataRequest{
        UserId: input.UserID,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get user health data: %v", err)
    }

    return &model.UserAccountPayload{...
    }, nil
}
}
```

Рисунок 34 – Обработчики запросов с клиента

```
func (r *queryResolver) GetUserVisitRecords(ctx context.Context, input model.UserAccountData) ([]*model.VisitRecord, error) {
    noteConnIdx := findServiceIndex(r.GrpcServices, "note_service")
    noteClient := pbNote.NewVisitServiceClient(r.GrpcServices[noteConnIdx].Conn)

    response, err := noteClient.GetUserVisitRecords(ctx, &pbNote.GetUserVisitRecordsRequest{
        UserId: input.UserID,
    })
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get user personal info: %v", err)
    }

    visits := make([]*model.VisitRecord, len(response.GetVisitRecords()))
    for i, visit := range response.GetVisitRecords() {
        visits[i] = &model.VisitRecord{
            RecordNumber: visit.GetRecordNumber(),
            DoctorName:    visit.GetDoctorName(),
            Specialization: visit.GetSpecialization(),
            VisitDate:     visit.GetVisitDate(),
        }
    }
    // You, 2 месяца назад • Update crew and note services

    return visits, nil
}

// GetDoctors is the resolver for the getDoctors field.
func (r *queryResolver) GetDoctors(ctx context.Context) ([]*model.Doctor, error) {
    crewConnIdx := findServiceIndex(r.GrpcServices, "crew_service")
    crewClient := pbCrew.NewCrewServiceClient(r.GrpcServices[crewConnIdx].Conn)

    response, err := crewClient.GetDoctors(ctx, &pbCrew.GetDoctorsRequest{})
    if err != nil {
        fmt.Println(err)
        return nil, fmt.Errorf("graphql: could not get list of doctors: %v", err)
    }

    doctors := make([]*model.Doctor, len(response.GetDoctors()))
    for i, doc := range response.GetDoctors() {
        doctors[i] = &model.Doctor{
            DoctorID:      doc.GetDoctorId(),
            DoctorName:     doc.GetDoctorName(),
            Specialization: doc.GetSpecialization(),
            ExperienceYears: int(doc.GetExperienceYears()),
        }
    }
    return doctors, nil
}
}
```

Рисунок 35 – Обработчики запросов с клиента

6.5 Разработка клиента

На рисунках 36-40 представлена основная логика клиента системы на React, отвечающего за предоставление пользователю интерфейса взаимодействия с системой.

```
import React from 'react';
import { BrowserRouter as Router, Route, Routes, Navigate } from 'react-router-dom';
import Home from '../home/Home';
import SignUp from '../signUp/SignUp';
import SignIn from '../signIn/SignIn';
import Account from '../account/Account';
import AccountNotes from '../account_notes/Account_notes';
import Notes from '../note/note';

You, 2 месяца назад • Change frontend/graphql and grpc

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/signup" element={<SignUp />} />
        <Route path="/signin" element={<SignIn />} />
        <Route path="/account" element={<Account />} />
        <Route path="/account/notes" element={<AccountNotes />} />
        <Route path="/logout" element={<Account />} />
        <Route path="/notes" element={<Notes />} />
        <Route path="*" element={<Navigate to="/" />} /> {}
      </Routes>
    </Router>
  );
}

export default App;
```

Рисунок 36 – Маршрутизатор конечных точек

```
function Account() {
  const userId = Cookies.get('userId');
  const { data, loading, error } = useQuery(GET_USER_INFO_QUERY, {
    variables: { input: { userId } },
    context: {
      uri: 'http://localhost:4000/graphql/account',
      credentials: 'include',
    },
  });

  const [updateUserHealthData] = useMutation(UPDATE_USER_HEALTH_DATA_MUTATION, {
    context: {
      uri: 'http://localhost:4000/graphql/account/update',
      credentials: 'include',
    },
  });

  const [formState, setFormState] = useState({});

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormState({
      ...formState,
      [name]: value ? roundToOneDecimal(parseFloat(value)) : '',
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    You, 2 месяца назад • Major changes for account

    const defaultData = data.getUserInfo;

    const input = {
      userId,
      age: formState.age !== undefined ? formState.age : roundToOneDecimal(defaultData.age),
      height: formState.height !== undefined ? formState.height : roundToOneDecimal(defaultData.height),
      weight: formState.weight !== undefined ? formState.weight : roundToOneDecimal(defaultData.weight),
      pulse: formState.pulse !== undefined ? formState.pulse : roundToOneDecimal(defaultData.pulse),
      pressure: formState.pressure !== undefined ? formState.pressure : roundToOneDecimal(defaultData.pressure),
    };

    console.log('Submitting:', input);
    updateUserHealthData({ variables: { input } }).then(() => {
      window.location.reload();
    })
    .catch((err) => {
      console.error('Ошибка обновления данных:', err);
    });
  };
}
```

Рисунок 37 – Код для страницы аккаунта пользователя

```
function VisitRecords() {
  const userId = Cookies.get('userId');
  const { data, loading, error } = useQuery(GET_USER_VISIT_RECORDS_QUERY, {
    variables: { input: { userId } },
    context: {
      uri: 'http://localhost:4000/graphql/account/notes',
      credentials: 'include',
    },
  });

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  return (
    <div>
      <h1>Visit Records</h1>
      {data.getUserVisitRecords.length === 0 ? (
        <p>No visit records found.</p>
      ) : (
        <table>
          <thead>
            <tr>
              <th>Record Number</th>
              <th>Doctor Name</th>
              <th>Specialization</th>
              <th>Visit Date</th>
            </tr>
          </thead>
          <tbody>
            {data.getUserVisitRecords.map((record) => (
              <tr Key={record.recordNumber}>
                <td>{record.recordNumber}</td>
                <td>{record.doctorName}</td>
                <td>{record.specialization}</td>
                <td>{record.visitDate}</td>
              </tr>
            ))}
          </tbody>
        </table>
      )}
      <Link to="/account">Back to Account</Link>
    </div>
  );
}

export default VisitRecords;
```

Рисунок 38 – Код для страницы записей пользователя

```
const DoctorsList = () => {
  const { loading, error, data } = useQuery(GET_DOCTORS, {
    context: {
      uri: 'http://localhost:4000/graphql/notes',
      credentials: 'include',
    },
  });

  const [addVisitRecord] = useMutation(ADD_VISIT_RECORD, {
    context: {
      uri: 'http://localhost:4000/graphql/notes/add',
      credentials: 'include',
    },
  });

  if (!userId) {
    return <p>Не удалось получить идентификатор пользователя из cookie.</p>;
  }

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  const handleRecordVisit = (doctorName, specialization) => {
    let visitDate = new Date();
    visitDate.setDate(visitDate.getDate() + 3);

    const formattedVisitDate = visitDate.toLocaleDateString('ru-RU', {
      year: 'numeric',
      month: '2-digit',
      day: '2-digit'
    });

    addVisitRecord({
      variables: {
        input: {
          userId,
          doctorName,
          specialization,
          visitDate: formattedVisitDate,
        },
      },
    })
      .then(() => {
        alert("Запись на прием успешно добавлена!");
      })
      .catch((err) => {
        console.error('Ошибка при добавлении записи:', err);
      });
  };
}
```

Рисунок 39 – Код для страницы записи на прием

```

function SignIn() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [signIn, { loading, error }] = useMutation(SIGN_IN_MUTATION, {
    context: {
      uri: 'http://localhost:4000/graphql/signin'
    }
  });
  const navigate = useNavigate();

  const handleSubmit = (e) => {
    e.preventDefault();
    signIn({ variables: { input: { username, password } } })
      .then((response) => {
        const { userId, role } = response.data.signIn;

        // Сохраняем в куки
        Cookies.set('userId', userId, { expires: 7 });
        Cookies.set('role', role, { expires: 7 });

        // Перенаправление после успешного входа
        navigate('/account');
      })
      .catch((err) => {
        console.error('Error during sign in:', err);
      });
  });

  return (
    <div>
      <h1>Login</h1>
      <Form onSubmit={handleSubmit}>
        <input
          type="text"
          value={username}
          placeholder="Username"
          onChange={(e) => setUsername(e.target.value)}
        />
        <input
          type="password"
          value={password}
          placeholder="Password"
          onChange={(e) => setPassword(e.target.value)}
        />
        <button type="submit" disabled={loading}>
          {loading ? 'Logging In...' : 'Login'}
        </button>
      </Form>
    </div>
  );
}

```

Рисунок 40 – Код для страницы входа в аккаунт

6.6 Разворачивание

Также для каждого сервиса написан Dockerfile для создания образа сервиса. Чтобы развернуть разработанную систему необходимо прописать файл docker-compose.yml, представленный на рисунках 41-43, который используется утилитой Docker Compose, который создает моноконтейнер, содержащий в себе контейнеры всех сервисов, баз данных и брокера сообщений, созданные из собранных заранее Docker образов.

```

version: '3.8'

services:
  react-client:
    build: ./client
    ports:
      - "80:80"
    depends_on:
      - graphql-server

  graphql-server:
    build: ./internal/services/graphql
    ports:
      - "4000:4000"
    depends_on:
      - auth_service
      - health_service
    networks:
      - microservices-network

  auth_service:
    build: ./internal/services/auth_service
    ports:
      - "5101:5101"
    depends_on:
      - auth_service-db
      - rabbitmq
    networks:
      - microservices-network
      - auth-network

  auth_service-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: usendb
    ports:
      - "5432:5432"
    volumes:
      - auth_service-db-data:/var/lib/postgresql/data
    networks:
      - auth-network

```

Рисунок 41 – Содержимое файла docker-compose.yml

```

services:
  health_service:
    ports:
      - "5102:5102"
    depends_on:
      - health_service-db
      - rabbitmq
    networks:
      - microservices-network
      - health-network

  health_service-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: healthdb
    ports:
      - "5433:5432"
    volumes:
      - health_service-db-data:/var/lib/postgresql/data
    networks:
      - health-network

  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: user
      RABBITMQ_DEFAULT_PASS: password
    volumes:
      - rabbitmq_data:/var/lib/rabbitmq
    networks:
      - microservices-network

```

Рисунок 42 – Содержимое файла docker-compose.yml

```

services:
  note_service:
    build: ./internal/services/note_service
    ports:
      - "5103:5103"
    depends_on:
      - note_service-db
    networks:
      - microservices-network
      - note-network

  note_service-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: notedb
    ports:
      - "5434:5432"
    volumes:
      - note_service-db-data:/var/lib/postgresql/data
    networks:
      - note-network

  crew_service:
    build: ./internal/services/crew_service
    ports:
      - "5104:5104"
    depends_on:
      - crew_service-db
    networks:
      - microservices-network
      - crew-network

  crew_service-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: crewdb
    ports:
      - "5435:5432"
    volumes:
      - crew_service-db-data:/var/lib/postgresql/data
    networks:
      - crew-network

```

Рисунок 43 – Содержимое файла docker-compose.yml

```

volumes:
  auth_service-db-data:
  health_service-db-data:
  note_service-db-data:
  crew_service-db-data:
  rabbitmq_data:

networks:
  microservices-network:
  auth-network:
  note-network:
  crew-network:
  health-network:
  driver: bridge

```

Рисунок 44 – Содержимое файла docker-compose.yml

На рисунках 45-47 показаны этапы сборки образов сервисов системы, создание необходимой инфраструктуры и контейнеров из образов на локальном сервере.

```
PS D:\MedApp> docker-compose up
times="2024-11-10T20:47:21+03:00" level=warning msg="D:\MedApp\docker-compose.yml: 'version' is obsolete"
[+] Running 11/6
  ✓ Network medapp_note-network          Created                                0.0s
  ✓ Network medapp_microservices-network Created                                0.0s
  ✓ Network medapp_crew-network          Created                                0.0s
[+] Running 16/16 default                Created                                0.0s
  ✓ Network medapp_note-network          Created                                0.0s
  ✓ Network medapp_microservices-network Created                                0.0s
  ✓ Network medapp_crew-network          Created                                0.0s
  ✓ Network medapp_default               Created                                0.0s
  ✓ Network medapp_health-network        Created                                0.1s
  ✓ Network medapp_auth-network          Created                                0.0s
  ✓ Volume "medapp_rabbitmq_data"        Created                                0.0s
[+] Running 20/20 crew_service-db-data  Created                                0.0s
  ✓ Network medapp_note-network          Created                                0.0s
  ✓ Network medapp_microservices-network Created                                0.0s
  ✓ Network medapp_crew-network          Created                                0.0s
  ✓ Network medapp_default               Created                                0.0s
  ✓ Network medapp_health-network        Created                                0.1s
  ✓ Network medapp_auth-network          Created                                0.0s
  ✓ Volume "medapp_rabbitmq_data"        Created                                0.0s
  ✓ Volume "medapp_crew_service-db-data" Created                                0.0s
  ✓ Volume "medapp_note_service-db-data" Created                                0.0s
[+] Running 21/21 note_service-db-data  Created                                0.0s
  ✓ Network medapp_note-network          Created                                0.0s
  ✓ Network medapp_microservices-network Created                                0.0s
  ✓ Network medapp_crew-network          Created                                0.0s
  ✓ Network medapp_default               Created                                0.0s
  ✓ Network medapp_health-network        Created                                0.1s
  ✓ Network medapp_auth-network          Created                                0.0s
  ✓ Volume "medapp_rabbitmq_data"        Created                                0.0s
  ✓ Volume "medapp_crew_service-db-data" Created                                0.0s
  ✓ Volume "medapp_note_service-db-data" Created                                0.0s
  ✓ Volume "medapp_health_service-db-data" Created                                0.0s
  ✓ Volume "medapp_auth_service-db-data" Created                                0.0s
[+] Running 22/22 health_service-db-data Created                                0.0s
  ✓ Network medapp_note-network          Created                                0.0s
  ✓ Network medapp_microservices-network Created                                0.0s
  ✓ Network medapp_crew-network          Created                                0.0s
  ✓ Network medapp_default               Created                                0.0s
  ✓ Network medapp_health-network        Created                                0.1s
  ✓ Network medapp_auth-network          Created                                0.0s
  ✓ Volume "medapp_rabbitmq_data"        Created                                0.0s
  ✓ Volume "medapp_crew_service-db-data" Created                                0.0s
  ✓ Volume "medapp_note_service-db-data" Created                                0.0s
  ✓ Volume "medapp_health_service-db-data" Created                                0.0s
  ✓ Volume "medapp_auth_service-db-data" Created                                0.0s
  ✓ Container medapp-crew_service-db-1   Created                                0.1s
  ✓ Container medapp-health_service-db-1 Created                                0.1s
  ✓ Container rabbitmq                   Created                                0.1s
  ✓ Container medapp-auth_service-db-1   Created                                0.1s
  ✓ Container medapp-note_service-db-1   Created                                0.1s
  ✓ Container medapp-crew_service-1       Created                                0.1s
  ✓ Container medapp-note_service-1       Created                                0.1s
  ✓ Container medapp-health_service-1     Created                                0.1s
```

Рисунок 45 – Сборка образов и контейнеров

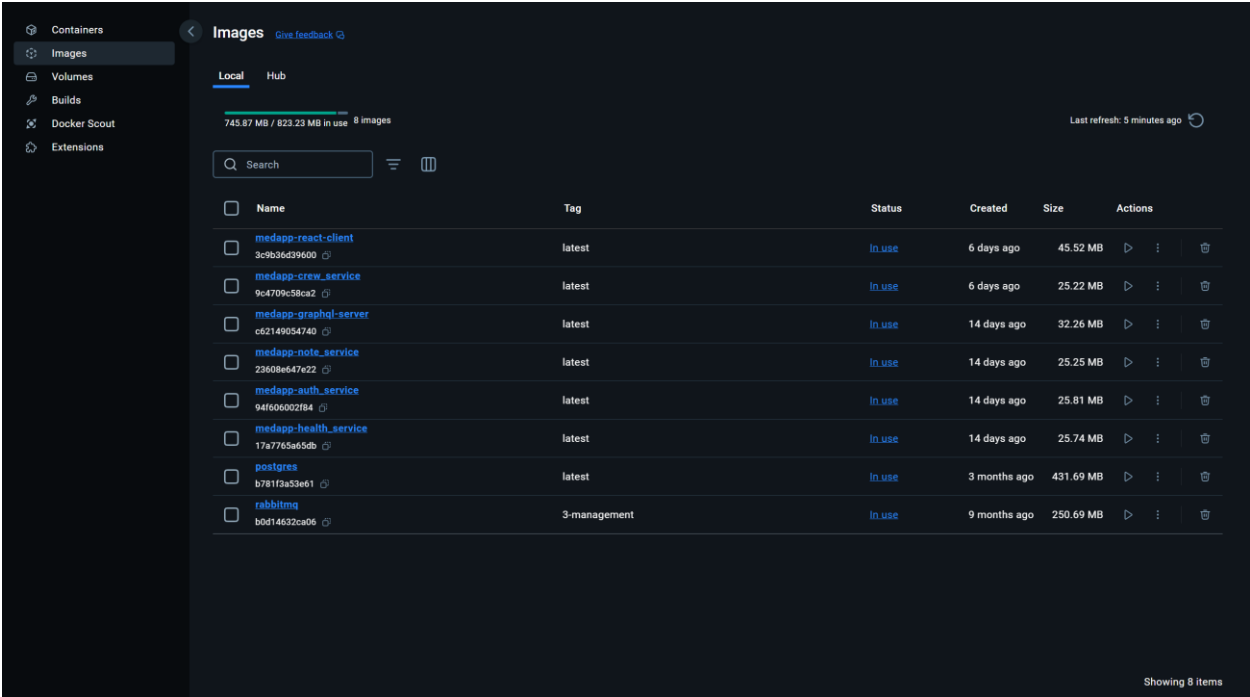


Рисунок 46 – Список образов компонентов системы в Docker Desktop

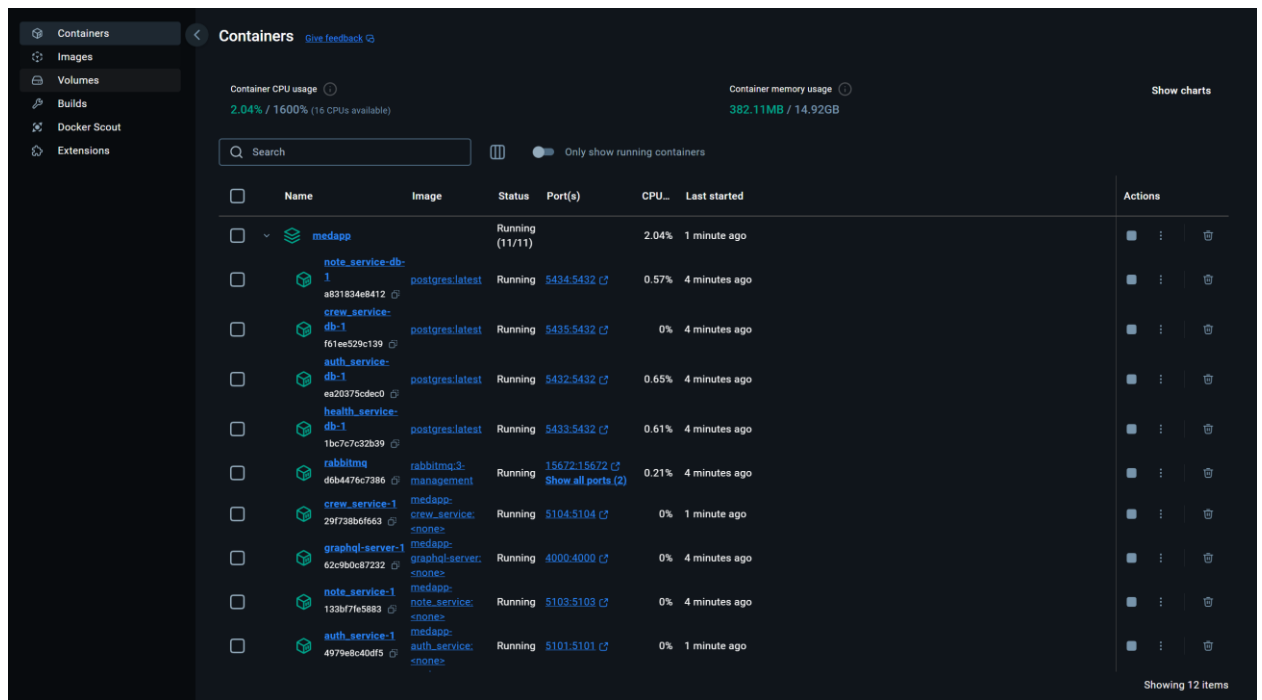


Рисунок 47 – Список контейнеров в Docker Desktop

Домашняя страница запущенного приложения представлена на рисунке 48.

Добро пожаловать на MedApp!

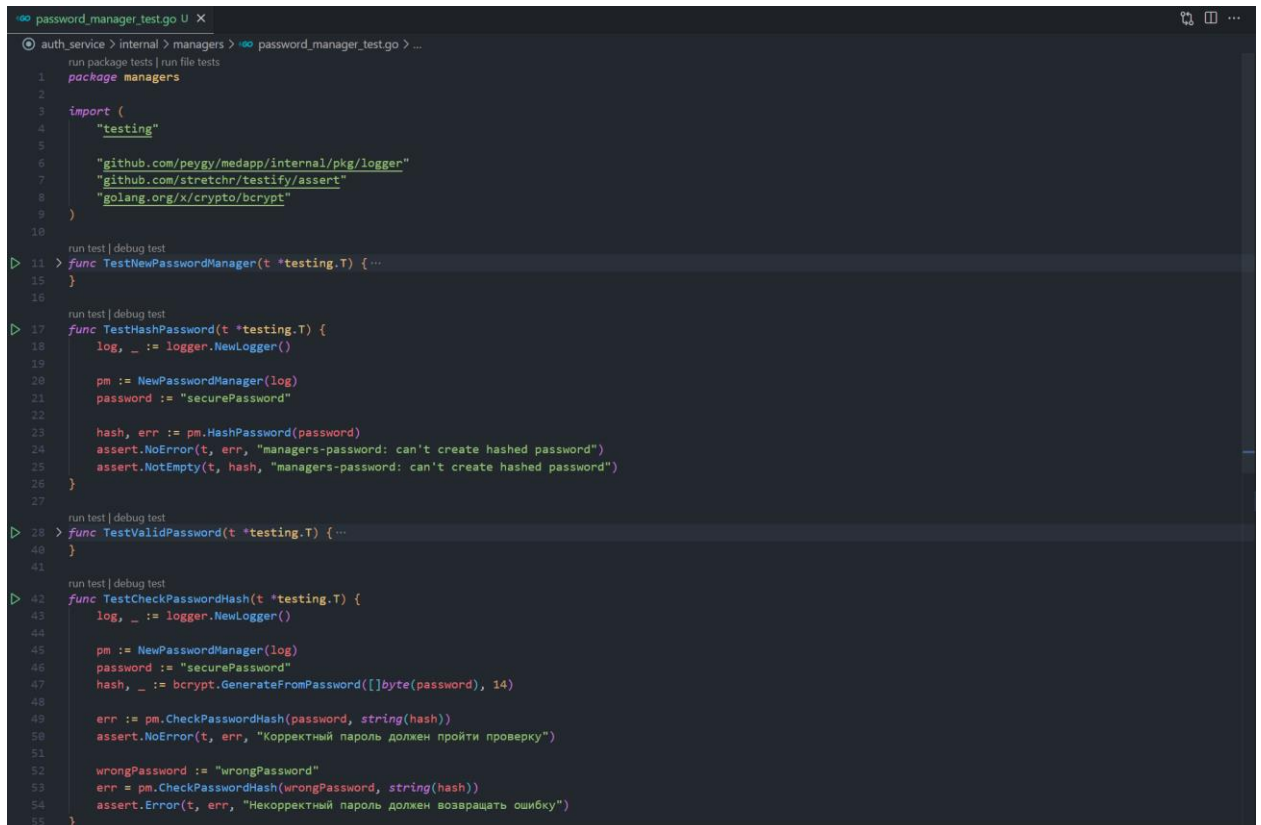
Регистрация аккаунта | Вход в аккаунт

Рисунок 48 – Домашняя страница приложения

7 ТЕСТИРОВАНИЕ

Проведем модульное тестирование кода разработанного приложения. Для этого к каждому сервису напишем ряд модульных тестов, для тестирования основного функционала каждого сервиса.

На рисунках 49-52 представлены тесты и результат их прохождения данных тестов в различных сценариях для сервиса AuthService.

A screenshot of a Go IDE window titled 'password_manager_test.go'. The code defines three test functions: 'TestNewPasswordManager', 'TestHashPassword', and 'TestCheckPasswordHash'. Each function uses 'testing.T' for assertions and a 'logger' for output. The first test creates a 'NewPasswordManager' with a logger and a password. The second test hashes a password and asserts that the hash is not empty. The third test checks a password hash, asserting that a correct password passes and an incorrect one returns an error.

```
1 package managers
2
3 import (
4     "testing"
5
6     "github.com/pevyg/medapp/internal/pkg/logger"
7     "github.com/stretchr/testify/assert"
8     "golang.org/x/crypto/bcrypt"
9 )
10
11 run test | debug test
12 > func TestNewPasswordManager(t *testing.T) { ...
13 }
14
15 run test | debug test
16
17 func TestHashPassword(t *testing.T) {
18     log, _ := logger.NewLogger()
19
20     pm := NewPasswordManager(log)
21     password := "securePassword"
22
23     hash, err := pm.HashPassword(password)
24     assert.NoError(t, err, "managers-password: can't create hashed password")
25     assert.NotEmpty(t, hash, "managers-password: can't create hashed password")
26 }
27
28 run test | debug test
29 > func TestValidPassword(t *testing.T) { ...
30 }
31
32 run test | debug test
33 func TestCheckPasswordHash(t *testing.T) {
34     log, _ := logger.NewLogger()
35
36     pm := NewPasswordManager(log)
37     password := "securePassword"
38     hash, _ := bcrypt.GenerateFromPassword([]byte(password), 14)
39
40     err := pm.CheckPasswordHash(password, string(hash))
41     assert.NoError(t, err, "Корректный пароль должен пройти проверку")
42
43     wrongPassword := "wrongPassword"
44     err = pm.CheckPasswordHash(wrongPassword, string(hash))
45     assert.Error(t, err, "Некорректный пароль должен возвращать ошибку")
46 }
```

Рисунок 49 – Тесты для PasswordManager

```
user_manager_test.go U X
auth_service > internal > managers > user_manager_test.go > TestGetUserByName_EmptyUsername

11
run test | debug test
12 func TestInsertUser(t *testing.T) {
13     db, mock, err := sqlmock.New()
14     assert.NoError(t, err)
15     defer db.Close()
16
17     log, _ := logger.NewLogger()
18     manager := NewUserManager(db, log)
19
20     user := UserRecord{
21         Username: "test_user",
22         Password: "hashed_password",
23     }
24
25     mock.ExpectQuery("INSERT INTO users").
26         WithArgs(user.Username, user.Password).
27         WillReturnRows(sqlmock.NewRows([]string{"id"}).AddRow("1"))
28
29     userId, err := manager.InsertUser(user)
30     assert.NoError(t, err)
31     assert.Equal(t, "1", userId)
32
33     err = mock.ExpectationsWereMet()
34     assert.NoError(t, err)
35 }
36
run test | debug test
37 > func TestInsertUser_Error(t *testing.T) { ...
60 }
61
run test | debug test
62 > func TestGetUserByName(t *testing.T) { ...
87 }
88
run test | debug test
89 > func TestGetUserById(t *testing.T) { ...
114 }
115
run test | debug test
116 func TestGetUserByName_EmptyUsername(t *testing.T) {
117     db, _, err := sqlmock.New()
118     assert.NoError(t, err)
119     defer db.Close()
120
121     log, _ := logger.NewLogger()
122     manager := NewUserManager(db, log)
123 }
```

Рисунок 50 – Тесты для UserManager

```
role_manager_test.go U X
auth_service > internal > managers > role_manager_test.go > TestAddRolesToUser

8
"github.com/pepygo/medapp/internal/pkg/logger"
9
"github.com/stretchr/testify/assert"
10
11
run test | debug test
12 > func TestNewRoleManager(t *testing.T) { ...
28 }
29
run test | debug test
30 func TestAddRolesToUser(t *testing.T) {
31     db, mock, err := sqlmock.New()
32     if err != nil {
33         t.Fatalf("failed to open mock db: %v", err)
34     }
35     defer db.Close()
36
37     log, _ := logger.NewLogger()
38
39     rm := &roleManager{db: db, log: log}
40
41     mock.ExpectQuery("SELECT id FROM roles WHERE role_name = `${id}`").
42         WithArgs("admin").
43         WillReturnRows(sqlmock.NewRows([]string{"id"}).AddRow("1"))
44
45     mock.ExpectExec("INSERT INTO users_roles (user_id, role_id) VALUES `${1}`, `${2}` ON CONFLICT DO NOTHING").
46         WithArgs("user1", "1").
47         WillReturnResult(sqlmock.NewResult(1, 1))
48
49     err = rm.AddRolesToUser("user1", "admin")
50     assert.Error(t, err)
51
52     err = mock.ExpectationsWereMet()
53     assert.Error(t, err)
54 }
55
run test | debug test
56 > func TestGetUserRole(t *testing.T) { ...
86 }
87
run test | debug test
88 > func TestDeleteRolesFromUser(t *testing.T) { ...
117 }
118
run test | debug test
119 > func TestRoleNotFound(t *testing.T) { ...
143 }
144 }
```

Рисунок 51 – Тесты для RoleManager

```

{"level":"info","ts":1731842520.6474943,"caller":"logger/logger.go:59","msg":"[INFO] User user1 role geted successfully"}
--- PASS: TestGetUserRole (0.00s)
=== RUN TestDeleteRolesFromUser
{"level":"info","ts":1731842520.6474943,"caller":"logger/logger.go:55","msg":"[INFO]Role-Delete: Transaction is begining"}
{"level":"error","ts":1731842520.6474943,"caller":"logger/logger.go:75","msg":"[ERROR] Can't starts transaction: call to database transaction Begin, was not expected, next expectation is: ExpectedQuery => expecting Query, QueryContext or QueryRow which:\n - matches sql: 'SELECT id FROM roles WHERE role_name = \\$(\\d+)'\\n - is with arguments:\n 0 - admin\n - should return rows:\n row 0 - [1]","stacktrace":"github.com/pegy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/pegy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.(*roleManager).DeleteRolesFromUser\n\tD:/MedApp/internal/services/auth_service/internal/managers/role_manager_test.go:111\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestDeleteRolesFromUser (0.00s)
=== RUN TestRoleNotFound
{"level":"info","ts":1731842520.6474943,"caller":"logger/logger.go:55","msg":"[INFO]Role-Add: Transaction is begining"}
{"level":"error","ts":1731842520.6474943,"caller":"logger/logger.go:75","msg":"[ERROR] Can't starts transaction: call to database transaction Begin, was not expected, next expectation is: ExpectedQuery => expecting Query, QueryContext or QueryRow which:\n - matches sql: 'SELECT id FROM roles WHERE role_name = \\$(\\d+)'\\n - is with arguments:\n 0 - nonexistent\n - should return error: sql: no rows in result set","stacktrace":"github.com/pegy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/pegy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.(*roleManager).AddRolesToUser\n\tD:/MedApp/internal/services/auth_service/internal/managers/role_manager_test.go:65\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.TestRoleNotFound\n\tD:/MedApp/internal/services/auth_service/internal/managers/role_manager_test.go:137\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestRoleNotFound (0.00s)
=== RUN TestInsertUser
{"level":"info","ts":1731842520.6488476,"caller":"logger/logger.go:55","msg":"[INFO]UserManager created"}
{"level":"info","ts":1731842520.6488476,"caller":"logger/logger.go:59","msg":"[INFO] User 1 created successfully"}
--- PASS: TestInsertUser (0.00s)
=== RUN TestInsertUser_Error
{"level":"info","ts":1731842520.6498547,"caller":"logger/logger.go:55","msg":"[INFO]UserManager created"}
{"level":"error","ts":1731842520.6498547,"caller":"logger/logger.go:75","msg":"[ERROR] Can't adds new user (test_user, hashed_password) to the database: insert error","stacktrace":"github.com/pegy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/pegy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.(*userManager).InsertUser\n\tD:/MedApp/internal/services/auth_service/internal/managers/user_manager_test.go:46\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.TestInsertUser_Error\n\tD:/MedApp/internal/services/auth_service/internal/managers/user_manager_test.go:54\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestInsertUser_Error (0.00s)
=== RUN TestGetUserByName
{"level":"info","ts":1731842520.6508546,"caller":"logger/logger.go:55","msg":"[INFO]UserManager created"}
{"level":"info","ts":1731842520.6508546,"caller":"logger/logger.go:59","msg":"[INFO] User test_user getted successfully"}
--- PASS: TestGetUserByName (0.00s)
=== RUN TestGetUserById
{"level":"info","ts":1731842520.6528587,"caller":"logger/logger.go:55","msg":"[INFO]UserManager created"}
{"level":"info","ts":1731842520.6528587,"caller":"logger/logger.go:59","msg":"[INFO] User 1 getted successfully"}
--- PASS: TestGetUserById (0.00s)
=== RUN TestGetUserByName_EmptyUsername
{"level":"info","ts":1731842520.6528587,"caller":"logger/logger.go:55","msg":"[INFO]UserManager created"}
{"level":"error","ts":1731842520.6528587,"caller":"logger/logger.go:75","msg":"[ERROR] Can't gets an user () from the database","stacktrace":"github.com/pegy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/pegy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers/user_manager_test.go:56\ngithub.com/pegy/medapp/internal/services/auth_service/internal/managers.TestGetUserByName_EmptyUsername\n\tD:/MedApp/internal/services/auth_service/internal/managers/user_manager_test.go:124\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetUserByName_EmptyUsername (0.00s)
PASS
ok      github.com/pegy/medapp/internal/services/auth_service/internal/managers      2.782s
PS D:\MedApp\internal\services\auth_service\internal\managers>

```

Рисунок 52 – Результат успешного прохождения тестов

На рисунках 53-54 представлены тесты и результат их прохождения данных тестов в различных сценариях для сервиса HealthService.

```

user-health_service_test.go X
health_service > internal > services > user-health_service_test.go > TestUpdateHealthDataById
run package tests | run file tests
1 package services
2
3 import (
4     "errors"
5     "testing"
6
7     "github.com/DATA-DOG/go-sqlmock"
8     "github.com/pegy/medapp/internal/pkg/logger"
9     "github.com/stretchr/testify/assert"
10 )
11
run test | debug test
12 > func TestInsertHealthData(t *testing.T) { ...
50 }
51
run test | debug test
52 > func TestGetHealthDataById(t *testing.T) { ...
93 }
94
run test | debug test
95 > func TestUpdateHealthDataById(t *testing.T) {
96     db, mock, err := sqlmock.New()
97     if err != nil {
98         t.Fatalf("Failed to create mock database connection: %v", err)
99     }
100     defer db.Close()
101
102     mockLogger, _ := logger.NewLogger()
103     service := NewHealthService(db, mockLogger)
104
run test | debug test
105     t.Run("Success", func(t *testing.T) {
106         healthData := HealthData{
107             Age:      30,
108             Height:   175.5,
109             Weight:   70.2,
110             Pulse:    72,
111             Pressure: "120/80",
112         }
113
114         mock.ExpectExec("UPDATE users_health_data SET age = $1, height = $2, weight = $3, pulse = $4, pressure = $5 WHERE userId = $6").
115             WithArgs(healthData.Age, healthData.Height, healthData.Weight, healthData.Pulse, healthData.Pressure, "123").
116             WillReturnResult(sqlmock.NewResult(1, 1))
117
118         updatedData, err := service.UpdateHealthDataById("123", healthData)
119     })

```

Рисунок 53 – Тесты для HealthService

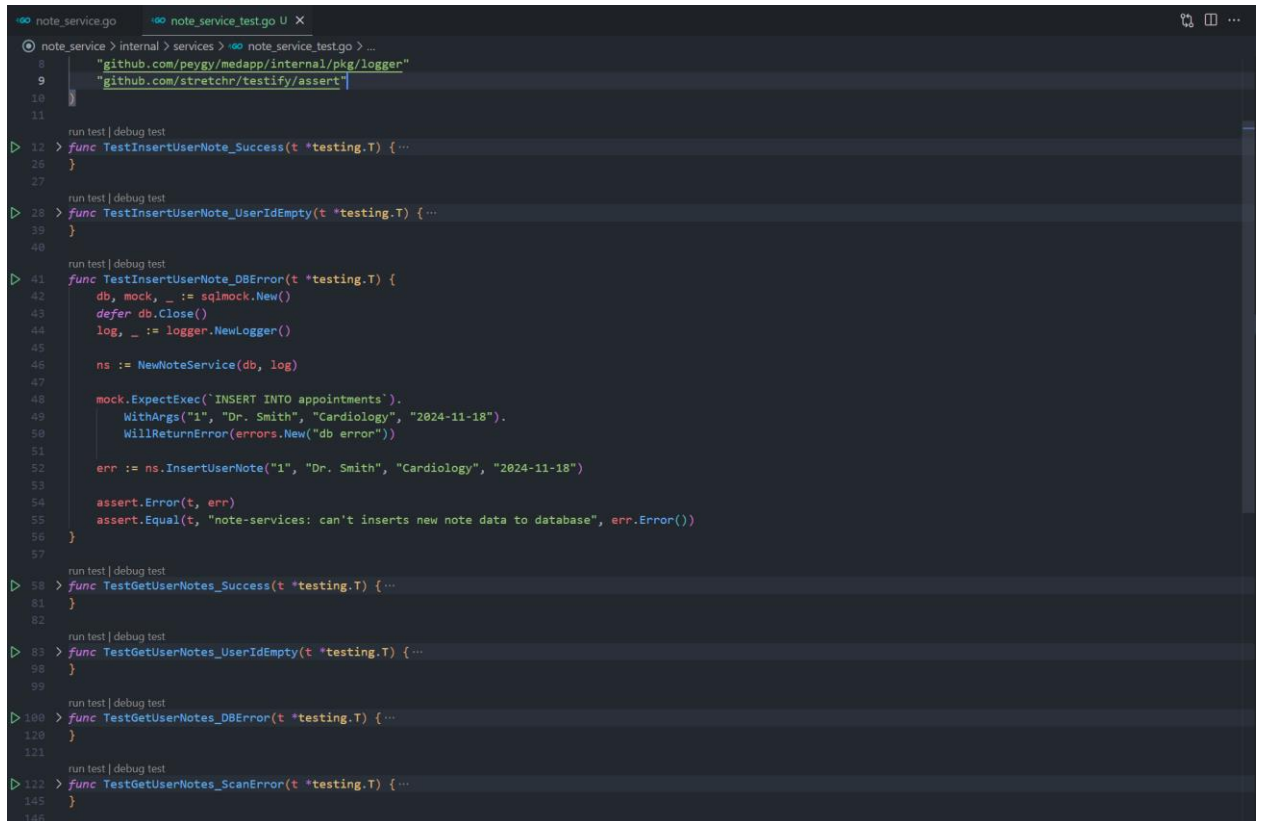
```

--- PASS: TestUpdateHealthDataByUserId (0.00s)
--- PASS: TestUpdateHealthDataByUserId/Success (0.00s)
--- PASS: TestUpdateHealthDataByUserId/Error:_empty_userId (0.00s)
--- PASS: TestUpdateHealthDataByUserId/Error:_empty_health_data (0.00s)
--- PASS: TestUpdateHealthDataByUserId/Error:_DB_update_error (0.00s)
PASS
ok      github.com/peygy/medapp/internal/services/health_service/internal/services 0.017s
PS D:\MedApp\internal\services\health_service\internal\services>

```

Рисунок 54 – Результат успешного прохождения тестов

На рисунках 55-56 представлены тесты и результат их прохождения данных тестов в различных сценариях для сервиса NoteService.



```

note_service.go  note_service_test.go U X
note_service> internal> services> note_service_test.go> ...
8      "github.com/peygy/medapp/internal/pkg/logger"
9      "github.com/stretchr/testify/assert"
10
11
12  run test | debug test
13  > func TestInsertUserNote_Success(t *testing.T) { ...
26  }
27
28  run test | debug test
29  > func TestInsertUserNote_UserIdEmpty(t *testing.T) { ...
39  }
40
41  run test | debug test
42  > func TestInsertUserNote_DBError(t *testing.T) {
43      db, mock, _ := sqlmock.New()
44      defer db.Close()
45      log, _ := logger.NewLogger()
46
47      ns := NewNoteService(db, log)
48
49      mock.ExpectExec("INSERT INTO appointments").
50          WithArgs("1", "Dr. Smith", "Cardiology", "2024-11-18").
51          WillReturnError(errors.New("db error"))
52
53      err := ns.InsertUserNote("1", "Dr. Smith", "Cardiology", "2024-11-18")
54
55      assert.Error(t, err)
56      assert.Equal(t, "note-services: can't inserts new note data to database", err.Error())
57  }
58
59  run test | debug test
60  > func TestGetUserNotes_Success(t *testing.T) { ...
81  }
82
83  run test | debug test
84  > func TestGetUserNotes_UserIdEmpty(t *testing.T) { ...
96  }
97
98  run test | debug test
99  > func TestGetUserNotes_DBError(t *testing.T) { ...
120  }
121
122  run test | debug test
123  > func TestGetUserNotes_ScanError(t *testing.T) { ...
145  }
146

```

Рисунок 55 – Тесты для NoteService

```

--- PASS: TestInsertUserNote_Success (0.00s)
=== RUN TestInsertUserNote_UserIdEmpty
{"level":"info","ts":"1731845114.703443","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"error","ts":"1731845114.703443","caller":"logger/logger.go:75","msg":"[ERROR] User id () is empty","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/note_service/internal/services.(*noteService).InsertUserNote\n\tD:/MedApp/internal/services/note_service/internal/services/note_service.go:37\ngithub.com/peygy/medapp/internal/services/note_service_test.go:41\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestInsertUserNote_UserIdEmpty (0.00s)
=== RUN TestInsertUserNote_DBError
{"level":"info","ts":"1731845114.7039583","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"error","ts":"1731845114.7039583","caller":"logger/logger.go:75","msg":"[ERROR] Can't inserts new note data for user (1) to database: db error","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/note_service/internal/services.(*noteService).InsertUserNote\n\tD:/MedApp/internal/services/note_service/internal/services/note_service.go:43\ngithub.com/peygy/medapp/internal/services/note_service_test.go:62\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestInsertUserNote_DBError (0.00s)
=== RUN TestGetUserNotes_Success
{"level":"info","ts":"1731845114.7044735","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"info","ts":"1731845114.7044735","caller":"logger/logger.go:59","msg":"[INFO] User (1) notes data retrieved successfully"}
--- PASS: TestGetUserNotes_Success (0.00s)
=== RUN TestGetUserNotes_UserIdEmpty
{"level":"info","ts":"1731845114.704989","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"error","ts":"1731845114.704989","caller":"logger/logger.go:75","msg":"[ERROR] User id () is empty","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/note_service/internal/services.(*noteService).GetUserNotes\n\tD:/MedApp/internal/services/note_service/internal/services/note_service.go:55\ngithub.com/peygy/medapp/internal/services/note_service_test.go:103\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetUserNotes_UserIdEmpty (0.00s)
=== RUN TestGetUserNotes_DBError
{"level":"info","ts":"1731845114.7055137","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"error","ts":"1731845114.7055137","caller":"logger/logger.go:75","msg":"[ERROR] Can't get notes data for user (1) from database: db error","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/note_service/internal/services.(*noteService).GetUserNotes\n\tD:/MedApp/internal/services/note_service/internal/services/note_service.go:62\ngithub.com/peygy/medapp/internal/services/note_service_test.go:125\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetUserNotes_DBError (0.00s)
=== RUN TestGetUserNotes_ScanError
{"level":"info","ts":"1731845114.7060287","caller":"logger/logger.go:55","msg":"[INFO]NoteService created"}
{"level":"error","ts":"1731845114.7065449","caller":"logger/logger.go:75","msg":"[ERROR] Error scanning note data for user (1): sql: Scan error on column index 3, name \"visit_date\": converting NULL to string is unsupported","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/note_service/internal/services.(*noteService).GetUserNotes\n\tD:/MedApp/internal/services/note_service/internal/services/note_service_test.go:150\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetUserNotes_ScanError (0.00s)
PASS
ok      github.com/peygy/medapp/internal/services/note_service/internal/services    0.016s
PS D:\MedApp\internal\services\note_service\internal\services>

```

Рисунок 56 – Результат успешного прохождения тестов

На рисунках 57-58 представлены тесты и результат их прохождения данных тестов в различных сценариях для сервиса CrewService.

```

crew_service_test.go X
crew_service > internal > services > crew_service_test.go > TestGetAllCrew_Success
11
run test | debug test
12 func TestGetAllCrew_Success(t *testing.T) {
13     db, mock, _ := sqlmock.New()
14     defer db.Close()
15     log, _ := logger.NewLogger()
16
17     cs := NewCrewService(db, log)
18
19     rows := sqlmock.NewRows([]string{"id", "doctor_name", "specialization", "experience_years"}).
20         AddRow("1", "Dr. Smith", "Cardiology", 10)
21
22     mock.ExpectQuery("SELECT id, doctor_name, specialization, experience_years").
23         WillReturnRows(rows)
24
25     crews, err := cs.GetAllCrew()
26
27     assert.NoError(t, err)
28     assert.Len(t, crews, 1)
29     assert.Equal(t, "Dr. Smith", crews[0].DoctorName)
30     assert.Equal(t, 10, crews[0].ExperienceYears)
31 }
32
run test | debug test
33 > func TestGetAllCrew_DBError(t *testing.T) { ...
52 }
53
run test | debug test
54 > func TestGetAllCrew_ScanError(t *testing.T) { ...
76 }
77
run test | debug test
78 > func TestGetAllCrew_EmptyResult(t *testing.T) { ...
98 }
99

```

Рисунок 57 – Тесты для CrewService


```
PS D:\MedApp\internal\services\crew_service\internal\services> go test -v
=== RUN TestGetAllCrew_Success
{"level":"info","ts":"1731845413.1482072","caller":"logger/logger.go:55","msg":"[INFO]CrewService created"}
--- PASS: TestGetAllCrew_Success (0.00s)
=== RUN TestGetAllCrew_DBError
{"level":"info","ts":"1731845413.1482072","caller":"logger/logger.go:55","msg":"[INFO]CrewService created"}
{"level":"error","ts":"1731845413.1487365","caller":"logger/logger.go:75","msg":"[ERROR] Can't get crews data from database: db error","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-20240923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/crew_service/internal/services.(*crewService).GetAllCrew\n\tD:/MedApp/internal/services/crew_service/internal/services/crew_service.go:49\ngithub.com/peygy/medapp/internal/services/crew_service/internal/services.TestGetAllCrew_DBError\n\tD:/MedApp/internal/services/crew_service/internal/services/crew_service_test.go:50\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetAllCrew_DBError (0.00s)
=== RUN TestGetAllCrew_ScanError
{"level":"info","ts":"1731845413.149252","caller":"logger/logger.go:55","msg":"[INFO]CrewService created"}
{"level":"error","ts":"1731845413.149252","caller":"logger/logger.go:75","msg":"[ERROR] Error scanning crew data: sql: Scan error on column index 3, name \"experience_years\": convertin g NULL to int is unsupported","stacktrace":"github.com/peygy/medapp/internal/pkg/logger.(*appLogger).Errorf\n\tC:/Users/user/go/pkg/mod/github.com/peygy/medapp/internal/pkg@v0.0.0-202 40923200850-00768f9c631c/logger/logger.go:75\ngithub.com/peygy/medapp/internal/services/crew_service/internal/services.(*crewService).GetAllCrew\n\tD:/MedApp/internal/services/crew_se rvice/internal/services/crew_service.go:54\ngithub.com/peygy/medapp/internal/services/crew_service/internal/services.TestGetAllCrew_ScanError\n\tD:/MedApp/internal/services/crew_servi ce/internal/services/crew_service_test.go:74\ntesting.tRunner\n\tC:/Program Files/Go/src/testing/testing.go:1690"}
--- PASS: TestGetAllCrew_ScanError (0.00s)
=== RUN TestGetAllCrew_EmptyResult
{"level":"info","ts":"1731845413.1497803","caller":"logger/logger.go:55","msg":"[INFO]CrewService created"}
--- PASS: TestGetAllCrew_EmptyResult (0.00s)
PASS
ok      github.com/peygy/medapp/internal/services/crew_service/internal/services      0.014s
PS D:\MedApp\internal\services\crew_service\internal\services> []
```

Рисунок 58 – Результат успешного прохождения тестов

Также проведем ручные тесты на работоспособность приложения. Тестирование представлено на рисунках 59-61.

Регистрация нового аккаунта

TestUser	*****	Зарегистрироваться
----------	-------	--------------------

Рисунок 59 – Тестирование регистрации нового аккаунта

Список врачей

- Фамилия: Иван Иванов
Специализация: Терапевт
Опыт: 10 лет
Записаться на прием
- Фамилия: Мария Петрова
Специализация: Хирург
Опыт: 15 лет
Записаться на прием
- Фамилия: Алексей Смирнов
Специализация: Кардиолог
Опыт: 8 лет
Записаться на прием
- Фамилия: Ольга Кузнецова
Специализация: Педиатр
Опыт: 5 лет
Записаться на прием
- Фамилия: Дмитрий Сидоров
Специализация: Невролог
Опыт: 12 лет
Записаться на прием
- Фамилия: Иван Иванов
Специализация: Терапевт
Опыт: 10 лет
Записаться на прием

Уведомление от сайта localhost
Запись на прием успешно добавлена!

Закрыть

Рисунок 60 – Тестирование добавления новой записи

Ваши записи к врачам

Номер записи	ФИО врача	Специализация врача	Дата приема
237b3833-dc03-4a63-9ce8-187c2f4a168c	Иван Иванов	Терапевт	13.11.2024

[Вернуться в личный кабинет](#)

Рисунок 61 – Тестирование вывода всех записей пользователя

ЗАКЛЮЧЕНИЕ

В результате разработки приложения была выполнена цель и поставленные задачи, проанализированы предметная область, архитектура и инструментарий разрабатываемого приложения, разработано приложение с использованием выбранной архитектуры и средств разработки, проведено тестирование приложения модульным и ручным методами. Разработанное приложение работает стабильно и предоставляет интуитивно понятный функционал для пользователей.

Разработанное приложение было развернуто на локальном хостинге персонального компьютера. Тесты показали положительную оценку с точки зрения корректности и оптимизации использования приложения.

Для ознакомления разработанным приложением, представлены ссылка на программный код данного приложения – <https://github.com/Peygy/MedApp>.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Golang Documentation / [Электронный ресурс] // go.dev : [сайт]. — URL: <https://go.dev/doc> (дата обращения: 24.09.2024).
2. Gin Documentation / [Электронный ресурс] // gin-gonic.com : [сайт]. — URL: <https://gin-gonic.com/docs> (дата обращения: 24.09.2024).
3. PostgreSQL 16.2 Documentation / [Электронный ресурс] // postgresql.org : [сайт]. — URL: <https://www.postgresql.org/docs/current/> (дата обращения: 24.09.2024).
4. Docker Manuals / [Электронный ресурс] // <https://docs.docker.com/manuals/> : [сайт]. — URL: <https://docs.docker.com> (дата обращения: 24.09.2024).
5. ЕМИАС / [Электронный ресурс] // emias.info : [сайт]. — URL: <https://emias.info> (дата обращения: 22.10.2024).
6. SmartMed / [Электронный ресурс] // medsi.ru : [сайт]. — URL: <https://medsi.ru/lands/smartmed.php> (дата обращения: 22.10.2024).
7. MedScannet / [Электронный ресурс] // medscannet.ru : [сайт]. — URL: <https://medscannet.ru> (дата обращения: 22.10.2024).
8. Monolithic Architecture - System Design / [Электронный ресурс] // geeksforgeeks.org : [сайт]. — URL: <https://www.geeksforgeeks.org/monolithic-architecture-system-design> (дата обращения: 13.10.2024).
9. Клиент-серверная архитектура в картинках / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/articles/495698> (дата обращения: 13.10.2024).
10. Сервис-ориентированная архитектура (SOA) / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/vk/articles/342526> (дата обращения: 13.10.2024).
11. Просто о микросервисах / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380> (дата обращения: 13.10.2024).

12. Что же такое этот GraphQL? / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/articles/326986> (дата обращения: 24.09.2024).

13. Introduction to gRPC / [Электронный ресурс] // grpc.io : [сайт]. — URL: <https://grpc.io/docs/what-is-grpc/introduction> (дата обращения: 24.09.2024).

14. RabbitMQ Tutorials / [Электронный ресурс] // rabbitmq.com : [сайт]. — URL: <https://www.rabbitmq.com/tutorials> (дата обращения: 24.09.2024).

15. Язык диаграмм / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/skyeng/articles/702024> (дата обращения: 25.10.2024).

16. Микросервисная архитектура, ее паттерны проектирования и особенности / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/serverspace/articles/692916> (дата обращения: 13.10.2024).

17. Spring. Экспертное мнение о плюсах и минусах самого популярного Java-фреймворка / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/companies/slurm/articles/698780> (дата обращения: 25.09.2024).

18. Плюсы и минусы Django / [Электронный ресурс] // habr.com : [сайт]. — URL: <https://habr.com/ru/articles/473042> (дата обращения: 25.09.2024).

19. Что такое Node.js и зачем ее изучать / [Электронный ресурс] // sky.pro : [сайт]. — URL: <https://sky.pro/media/chto-takoe-node-js> (дата обращения: 25.09.2024).

20. AngularJS API Docs / [Электронный ресурс] // docs.angularjs.org : [сайт]. — URL: <https://docs.angularjs.org/api> (дата обращения: 24.09.2024).

21. Introduction Vue.js / [Электронный ресурс] // vuejs.org : [сайт]. — URL: <https://vuejs.org/guide/introduction.html> (дата обращения: 24.09.2024).

22. React Documentation / [Электронный ресурс] // react.dev : [сайт]. — URL: <https://react.dev/learn> (дата обращения: 24.09.2024).

23. MySQL Docs / [Электронный ресурс] // dev.mysql.com : [сайт]. — URL: <https://dev.mysql.com/doc> (дата обращения: 24.09.2024).

24. SQLite Documentation / [Электронный ресурс] // www.sqlite.org : [сайт]. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 24.09.2024).